

phenometrics

An R package for calculating Time of Emergence (ToE) metrics and indicators on phenological data

Abstract

This vignette walks through the basic functionality of the phenometrics package, as well as various customizations built into it and how/when to use them.

Contents

1	phenometrics basic functionality	3
2	Package Options and Advice for End-Users	18
2.1	Directionality	18
2.2	Threshold Values	21
2.3	Baseline Construction	26
2.4	Data Corrections and Considerations	45

1 phenometrics basic functionality

Unlike standard phenological analyses, which tend to focus on rates of change, phenometrics calculates Time of Emergence (ToE) metrics, which attempt to identify the exact point at which a species' phenology has exceeded its historical variability and emerged from its previous regime. The phenometrics package assumes input data is provided as a data frame, with standardized column names:

year: Year of the phenological event (numeric).

species: Unique species identifier. Can be numeric or character. Required for community but not species level functions.

event: Time at which the phenological event occurs, typically day of year (numeric).

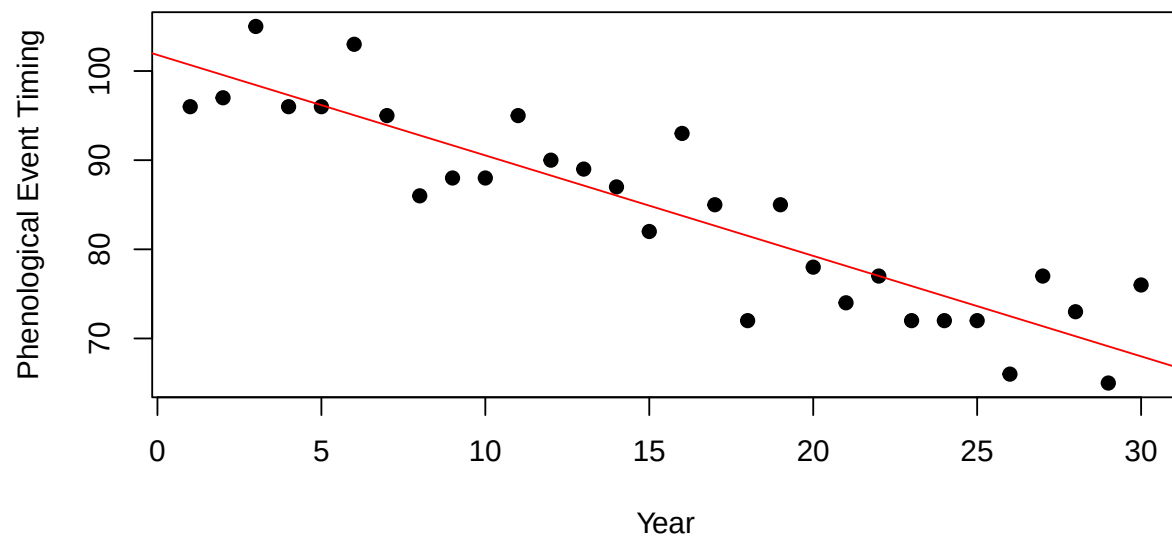
env: Environmental conditions (usually temperature) at which the phenological event occurs (numeric). Required for time of environmental emergence functions or community functions.

The basic worker function in the phenometrics package is `emp_tope()`, which calculates the Time of Phenological Emergence (ToPE) for a species using empirical methodology. ToPE only requires two columns in the input dataset: **year** and **event**.

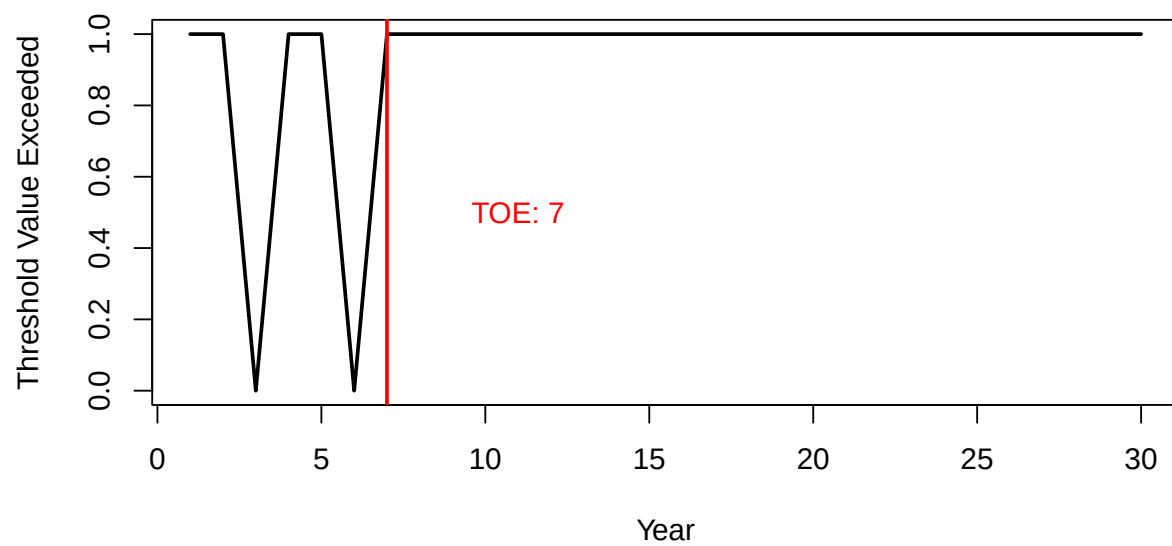
```
# library
library(phenometrics)
library(tidyverse)

# Set seed
set.seed(123)

# Create test dataset
year = seq(1,30,1)
event = round(rnorm(30, 100, 5)) - seq(1,30,1)
dataset = data.frame(year, event)
plot(dataset, xlab = "Year", ylab = "Phenological Event Timing", pch = 19)
abline(lm(data = dataset, event ~ year), col = 'red')
```



```
# Calculate empirical time of phenological emergence (ToPE)
head(emp_tope(dataset))
```



```
#>   year p emerged
#> 1    1 1      0
#> 2    2 1      0
```

```
#> 3      3 0      0
#> 4      4 1      0
#> 5      5 1      0
#> 6      6 0      0
```

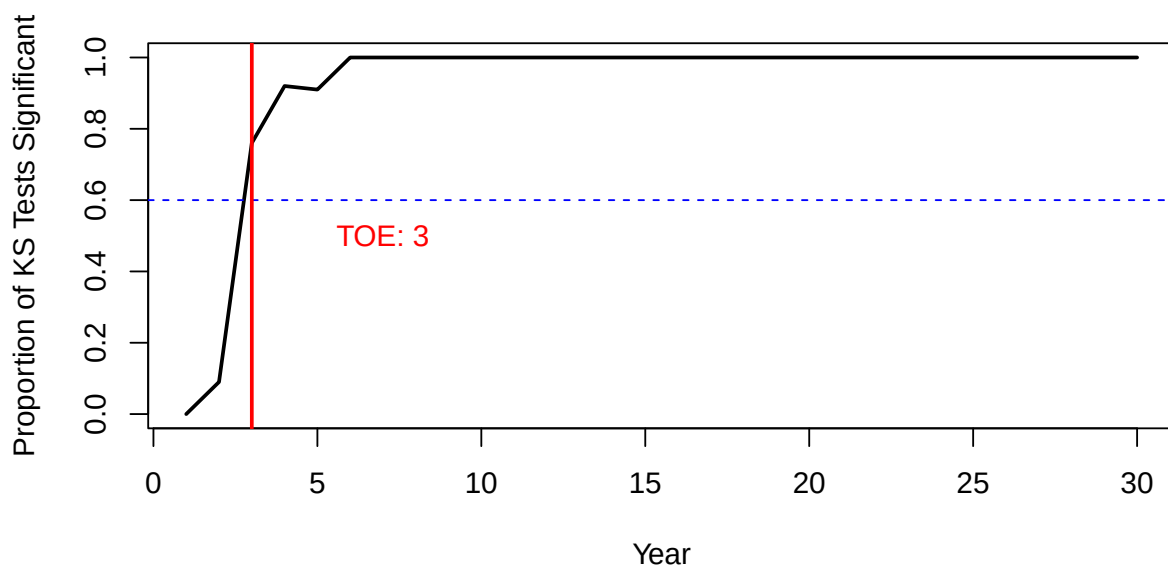
By default, this function outputs a data frame that contains the year, test value p (1 = positive test value, indicating that year has exceeded historical variability), emergence status (1 = emerged, 0 = not emerged). Emergence occurs once a certain number of consecutive years return positive test values (set to 5 by default), and the point of emergence is the first of those consecutive years (i.e. it is checking whether the emergence holds for the required time interval. This is also why the final returned data frame contains NAs in the final 4 years, as there are insufficient subsequent years to check for emergence). Unless told not to, it also generates a plot of test results against year, and if applicable, indicates the point at which emergence occurs.

Alternatively, `ks_tope()` calculates ToPE using statistical methodology via bootstrapped Kolmogorov-Smirnov tests. This requires there to be replicates within each year to serve as the sample size for the test; the example generated dataset has thirty replicates.

```
# Set seed
set.seed(123)

# Create test dataset
year = rep(seq(1,30,1),10)
event = round(rnorm(900, 100, 5)) - year
dataset = data.frame(year, event) %>% arrange(year)

# Calculate empirical time of phenological emergence (ToPE)
head(ks_tope(dataset))
```



```
#>   year    p emerged
#> 1     1 0.00      0
```

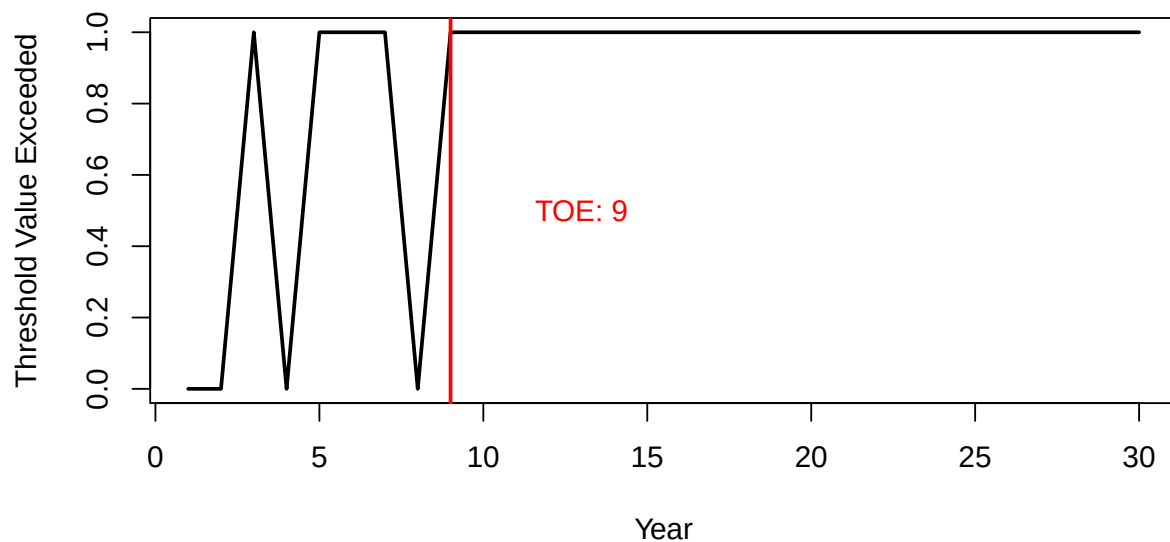
```
#> 2    2 0.09    0
#> 3    3 0.76    1
#> 4    4 0.92    1
#> 5    5 0.91    1
#> 6    6 1.00    1
```

To examine the environmental conditions a phenological event occurs at rather than its timing, use the mirror function to `emp_tope()` which is `emp_toe()` (and to `ks_tope()` is `ks_toe()`). We refer to this as the Time of Environmental Emergence (ToEE).

```
# Set seed
set.seed(123)

# Create test dataset
year = seq(1,30,1)
env = (rnorm(30, 15, 0.5)) + seq(0.1,3,0.1)
dataset = data.frame(year, env)

# Calculate empirical time of phenological emergence (ToPE)
head(emp_toe(dataset))
```



```
#>   year p emerged
#> 1    1 0      0
#> 2    2 0      0
#> 3    3 1      0
#> 4    4 0      0
#> 5    5 1      0
#> 6    6 1      0
```

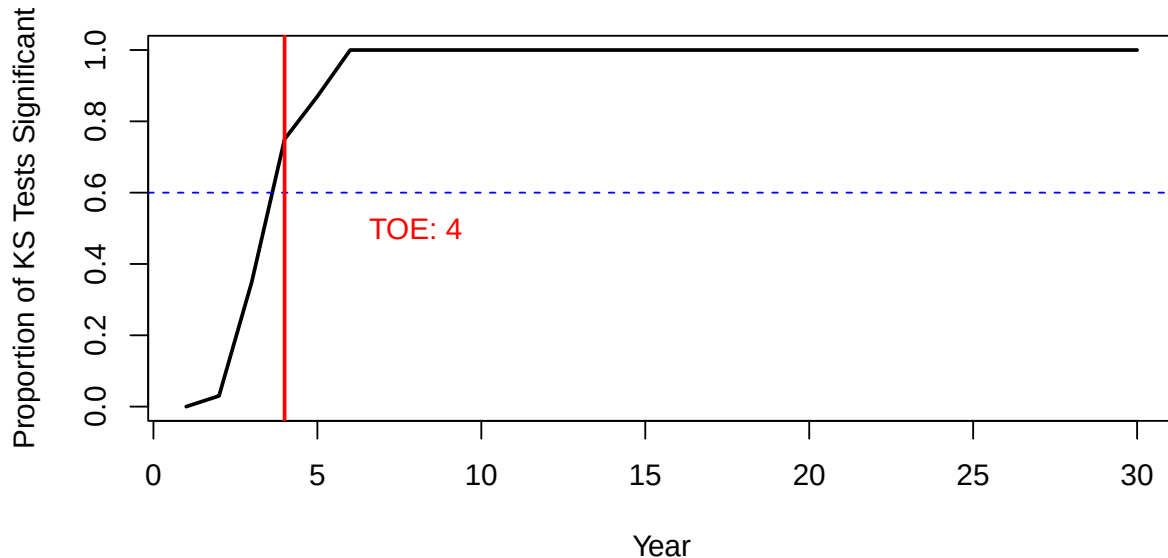
```
# Create test dataset
```

```

year = rep(seq(1,30,1),30)
env = (rnorm(900, 15, 0.5)) + rep(seq(0.1,3,0.1), 30)
dataset = data.frame(year, env) %>% arrange(year)

# Calculate empirical time of phenological emergence (ToPE)
head(ks_toee(dataset))

```



```

#>   year    p emerged
#> 1     1 0.00      0
#> 2     2 0.03      0
#> 3     3 0.35      0
#> 4     4 0.75      1
#> 5     5 0.87      1
#> 6     6 1.00      1

```

These functions perform the same analyses as `emp_tope()` and `ks_tope()`, but on the `env` column instead of the `event` column, and with the direction reversed (by default, ToPE functions look for declining trends in the time series, and ToEE functions look for increasing trends such as an increase in temperature at which a phenological event occurs - this behaviour can be changed using the `alt` argument). Together, ToPE and ToEE can be used to classify a species' phenological response to climate change. For example, if a species emerges in ToPE but not ToEE, then it is shifting its phenology in time but maintaining the environmental conditions at which that phenology occurs. Conversely, a species emerged in ToEE and not ToPE is doing the opposite: it is maintaining the timing of its phenology at the expense of changing the environmental conditions under which it occurs.

phenometrics also contains community-level functions that call `emp_tope()` and `emp_toee()` to perform many ToPE and ToEE calculations and then summarize the results across multi-species datasets. `community()` calculates ToPE and ToEE for all unique species in a dataset, and generates classifications based on the results. `class_by_species` and `class_by_year` further summarize and plot these results. They use empirical methodology by default - this can be changed by setting `method = 'statistical'`.

```

# Set seed
set.seed(123)

# Create 5 test species
dataset = NULL
rev = 5:1
for(i in 1:5){

  # Year
  year = seq(1,30,1)

  # Event timing
  event = round(rnorm(30, 100, 5)) - (seq(1,30,1)/i)

  # Environment
  env = (rnorm(30, 15, 0.5)) + (seq(0.1,3,0.1)/rev[i])

  # Species id
  species = i

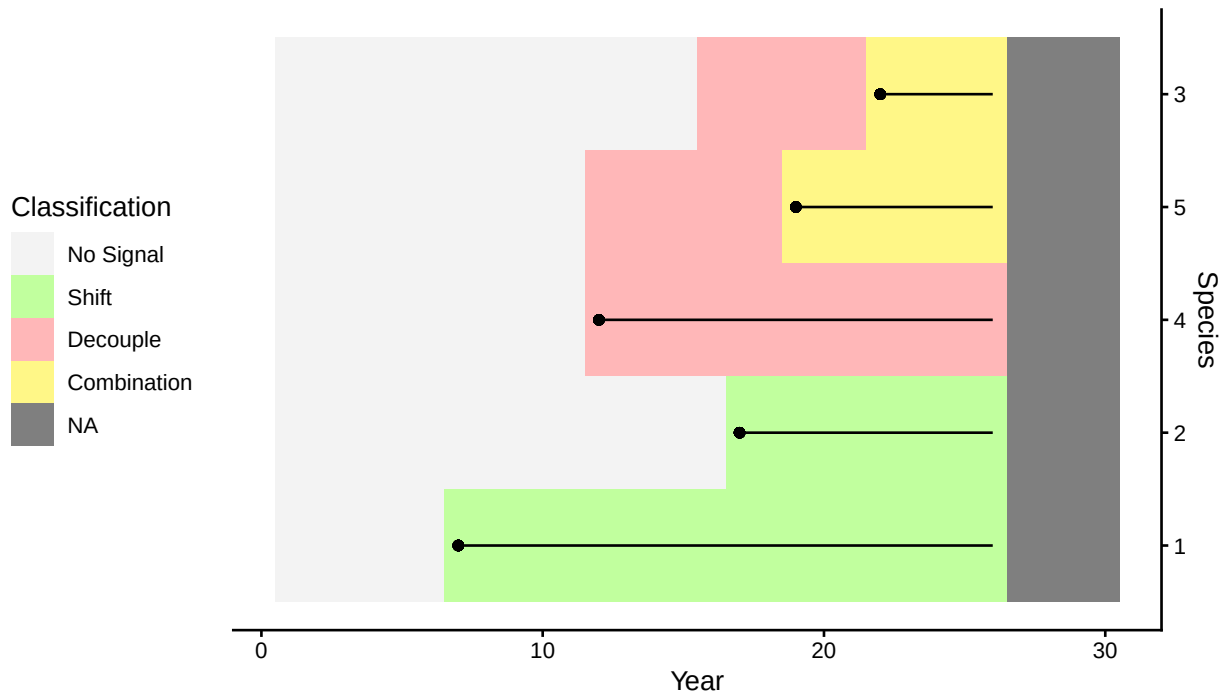
  # bind
  dataset = rbind(dataset, data.frame(year, species, event, env))

}

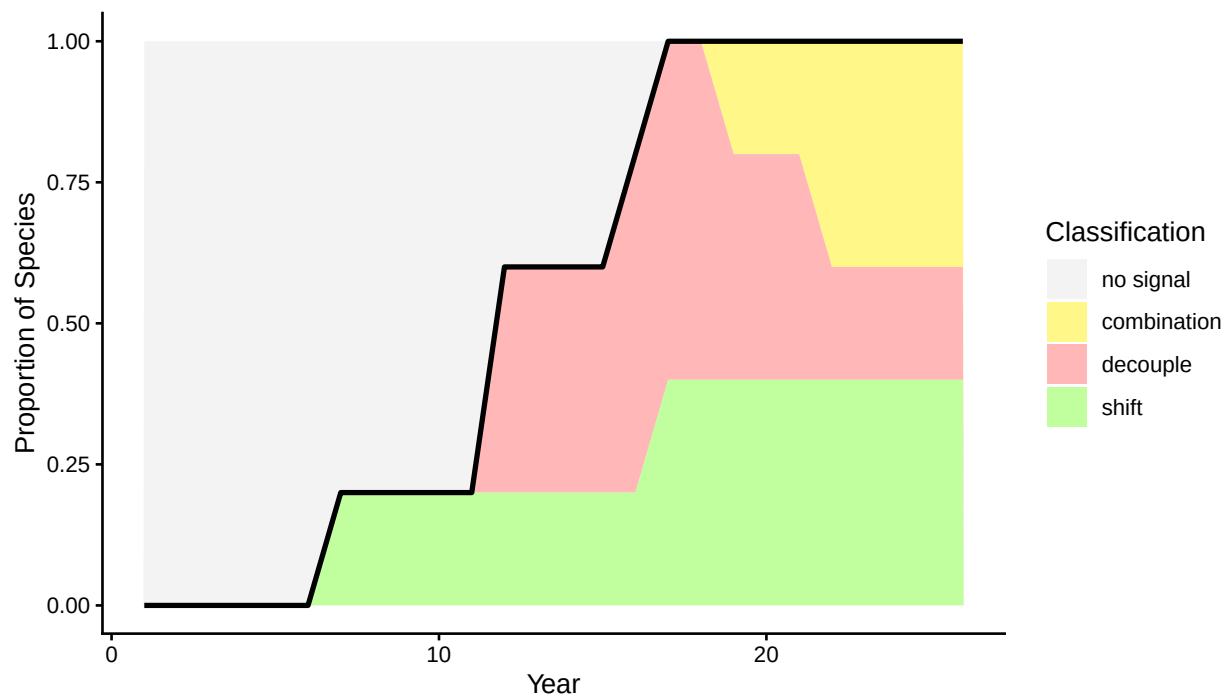
# Calculate empirical time of phenological emergence (ToPE)
comm = community(dataset)

# Summarize and plot results
cbs = class_by_species(comm)
#> Joining with `by = join_by(species)`

```

```
cby = class_by_year(comm)
```



Additionally, phenometrics can also use ToE methodology to determine the point at which phenological mismatch occurs between two species. We call this Time of Mismatch (ToM), and it can be calculated using the `emp_tom()` and `ks_tom()` functions.

```

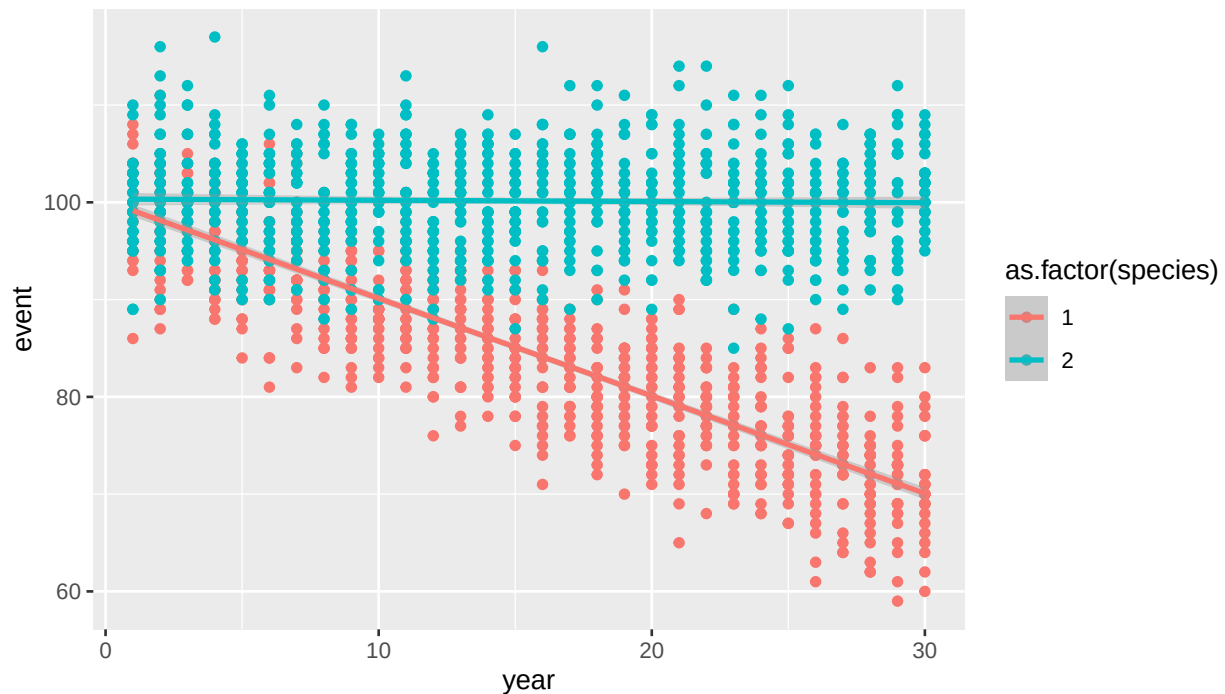
# Set seed
set.seed(123)

# Create test dataset 1
year = rep(seq(1,30,1), 30)
event = round(rnorm(900, 100, 5)) - year
species = 1
sp1 = data.frame(year, species, event) %>% arrange(year)

# Create test dataset 2
event = round(rnorm(900, 100, 5))
species = 2
sp2 = data.frame(year, species, event) %>% arrange(year)

# Plot species phenology time series
ggplot(rbind(sp1, sp2), aes(x = year, y = event, color = as.factor(species))) +
  geom_point() + stat_smooth(method = 'lm')
#> `geom_smooth()` using formula = 'y ~ x'

```

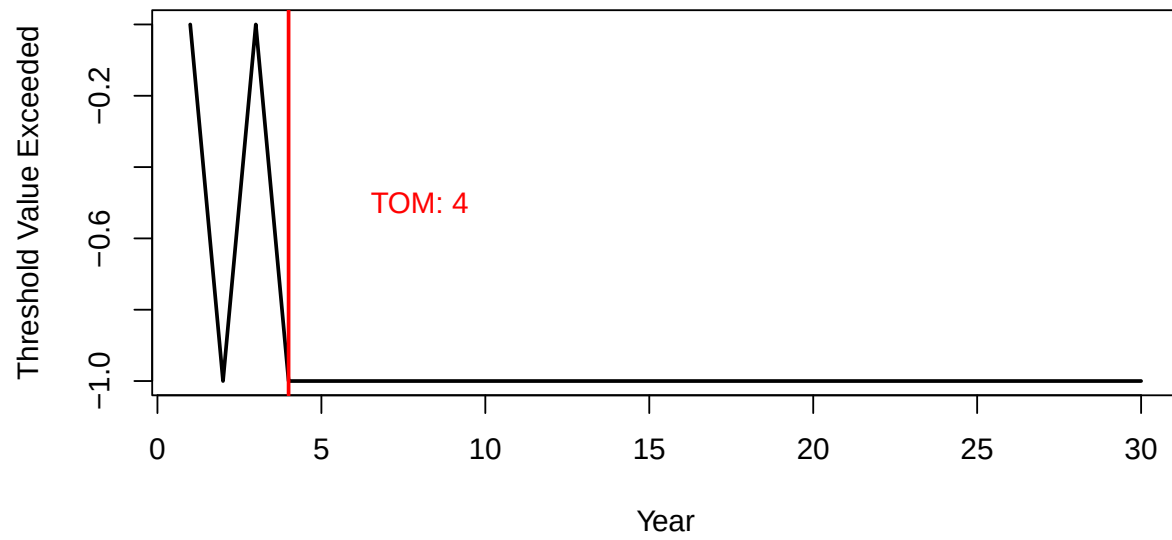


```

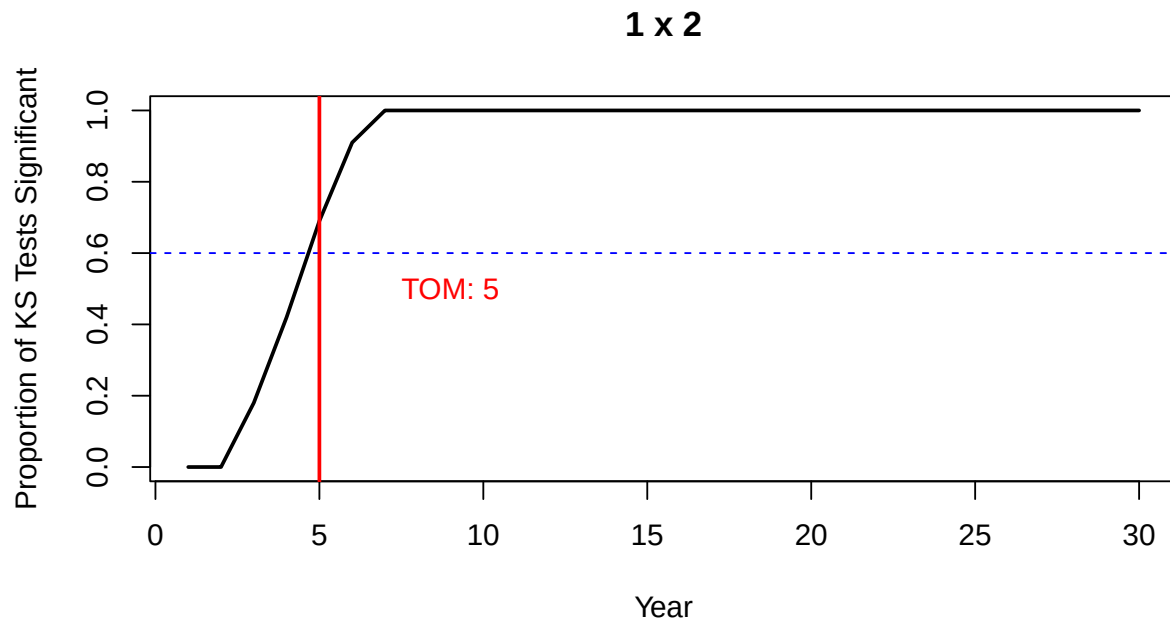
# Calculate empirical time of phenological emergence (ToPE)
head(emp_tom(sp1, sp2))

```

1 x 2



```
#>   year p emerged
#> 1    1 0      0
#> 2    2 -1     0
#> 3    3 0      0
#> 4    4 -1     1
#> 5    5 -1     1
#> 6    6 -1     1
head(ks_tom(sp1, sp2))
```



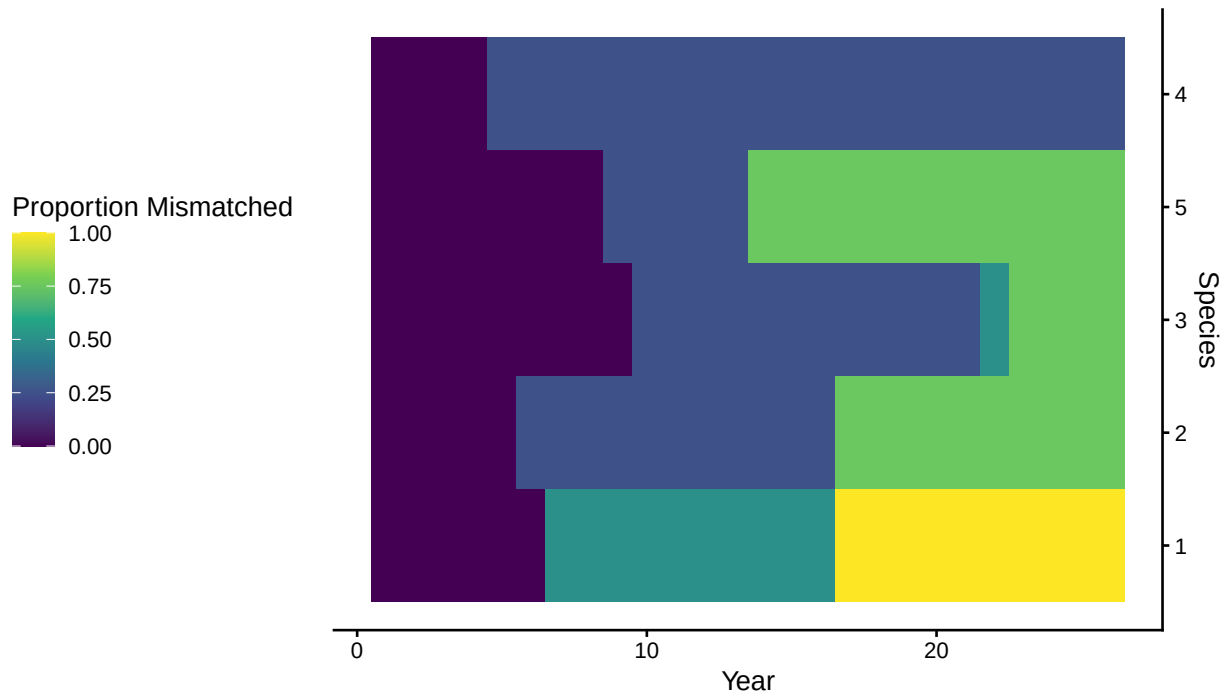
```
#>   year    p emerged
#> 1     1 0.00      0
#> 2     2 0.00      0
#> 3     3 0.18      0
#> 4     4 0.42      0
#> 5     5 0.69      1
#> 6     6 0.91      1
```

You'll notice ToM results take a similar form to the ToPE and ToEE results. The similarities don't end there - phenometrics also contains community-level wrapper functions for multi-species mismatch datasets. `comm_mismatch()`, `comm_mm_by_spp()`, and `comm_mm_by_year()` are the mismatch equivalents to the above ToPE/ToEE-based functions. As with `class_by_species` and `class_by_year`, they use empirical methodology by default but this can be changed by setting `method = 'statistical'`.

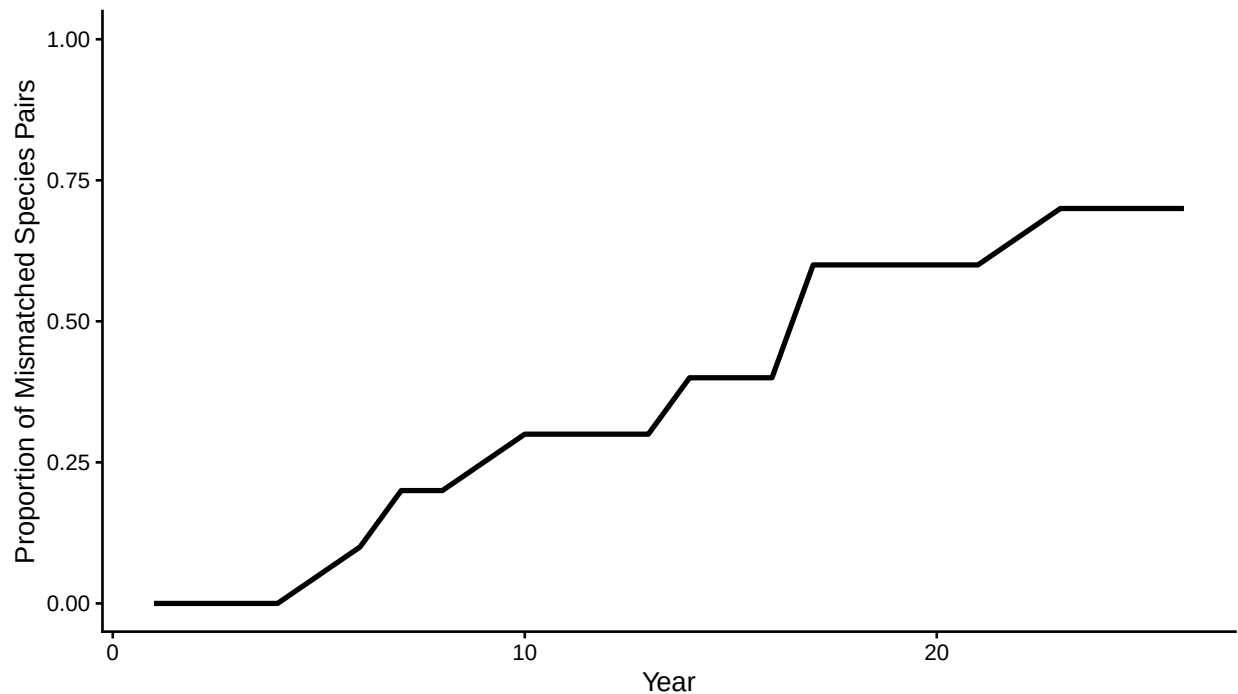
```
# Set seed
set.seed(123)

# Calculate empirical time of phenological emergence (ToPE)
comm_mm = comm_mismatch(dataset)

# Summarize and plot results
mm_cbs = comm_mm_by_spp(comm_mm)
```



```
mm_cby = comm_mm_by_year(comm_mm)
```



`comm_mismatch` only tests species against each other if their event timing values are within one standard deviation of each other to avoid frivolous matching behaviour. `comm_mm_by_spp()` calculates and plots the proportion of relevant species in the dataset with which each species is mismatched in each year, while `comm_mm_by_year()` averages these values each year to calculate the average mismatch proportion of the entire community each year.

The final function in the `phenometrics` package is `sim_ts()`, which is used to simulate future years of a phenology time series. It can be used to estimate the time of future emergence in cases where a trend exists, but emergence has not yet occurred. This function works by sampling points from the provided data and adding the linear trend. Sample size, variability, and linear trend are all extracted from the provided original time series and assumed to remain constant. `sim_ts()` can handle multi-species datasets, and can simulate one or both of `event` and `env`.

```
# Set seed
set.seed(123)

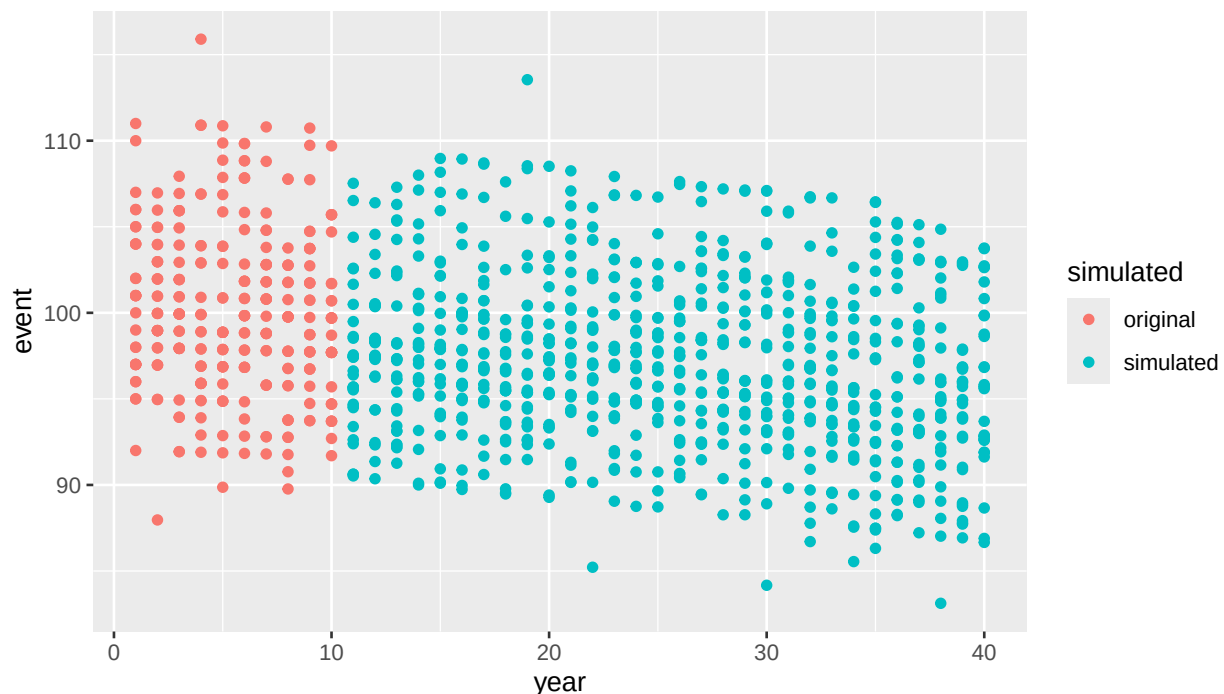
# Create test dataset
year = rep(seq(1,10,1), 10)
event = round(rnorm(300, 100, 5)) - (rep(seq(0,0.3,length.out = 10), 10))
species = 1
env = (rnorm(300, 15, 0.5))
dataset = data.frame(year, species, event, env); dataset = dataset[order(dataset$year),]

# Years to Simulate (30 more years)
years_sim = max(year+1):max(year+30)

# Run simulation
data_sim = sim_ts(dataset, years_sim)

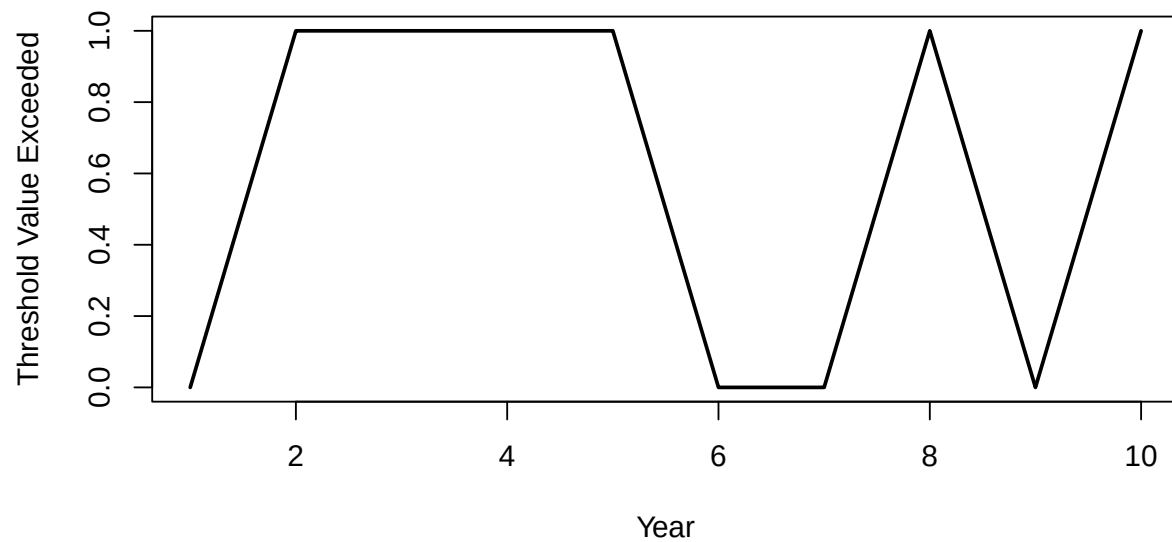
# Add simulation identifier
data_sim$simulated = ifelse(data_sim$year %in% years_sim, 'simulated', 'original')

# Plot
ggplot(data_sim, aes(x = year, y = event, color = simulated)) + geom_point()
```

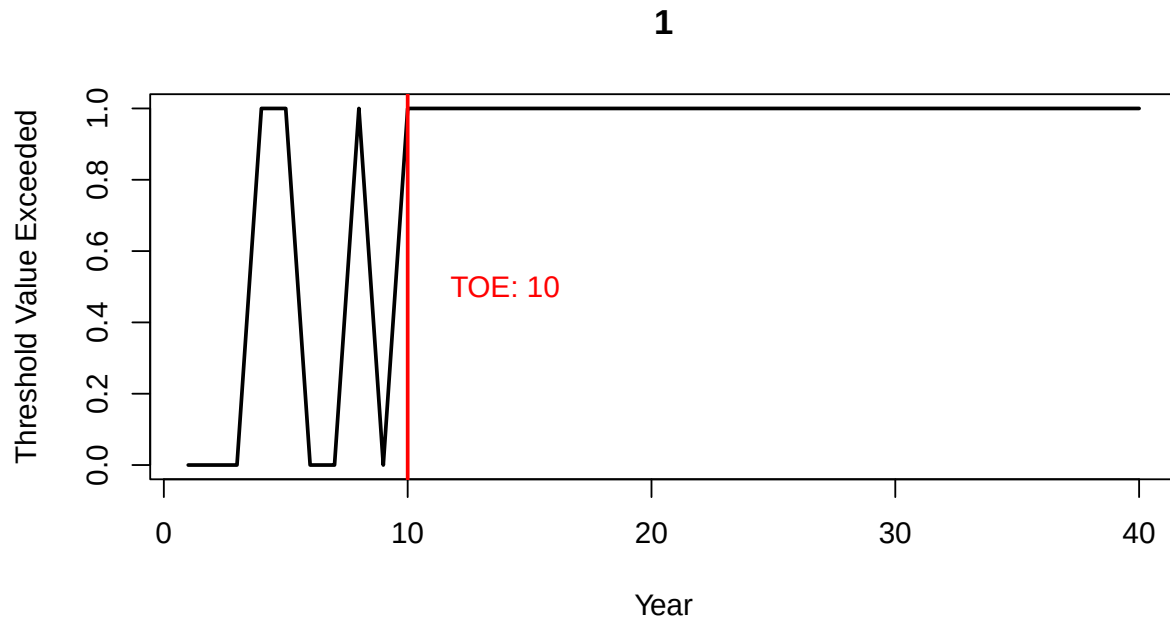


```
# Tests
test = emp_tope(dataset) # Emergence has not yet occurred
#> Warning in min(which(emerged == 1)): no non-missing arguments to min; returning
#> Inf
#> Warning in min(which(emerged == 1)): no non-missing arguments to min; returning
#> Inf
#> Warning in min(which(emerged == 1)): no non-missing arguments to min; returning
#> Inf
```

1



```
test = emp_tope(data_sim) # When emergence is predicted to occur based on simulated years
```



If using this functionality, we recommend recalculating many simulated time series and summarizing the resulting ToE values in order to buffer for simulation randomness. For example:

```
# 100 iterations of ToE calculations on simulated datasets
toe = c()
for(i in 1:100){

  # Run simulation
  data_sim = sim_ts(dataset, years_sim)

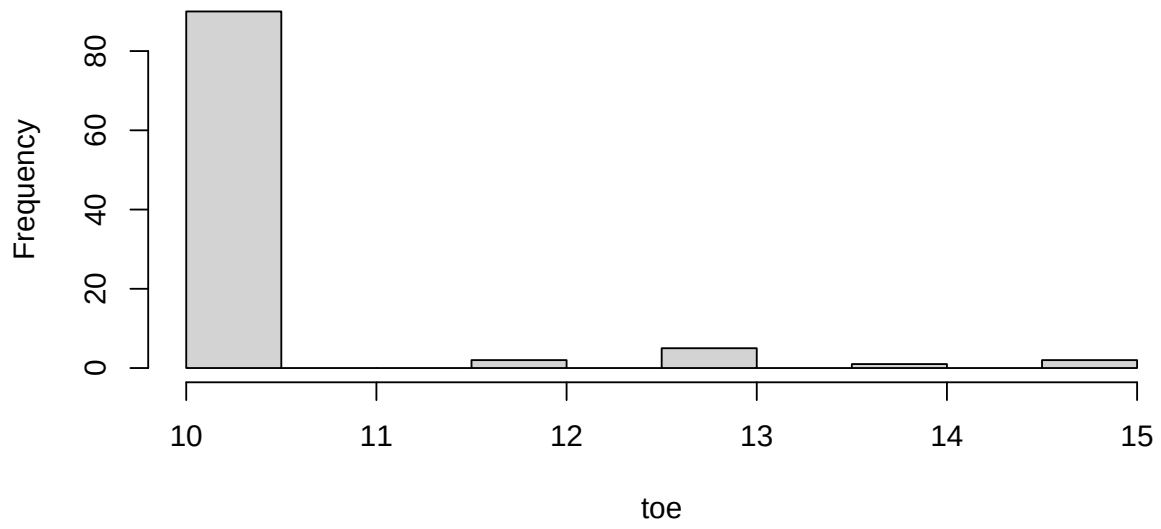
  # Calculate ToPE
  toe_yr = emp_tope(data_sim, plot = F)

  # ToE is first year with emerged == 1
  toe = c(toe, min(toe_yr$year[toe_yr$emerged==1], na.rm = T))

}

# Plot histogram of ToE values
hist(toe)
```


Histogram of toe



2 Package Options and Advice for End-Users

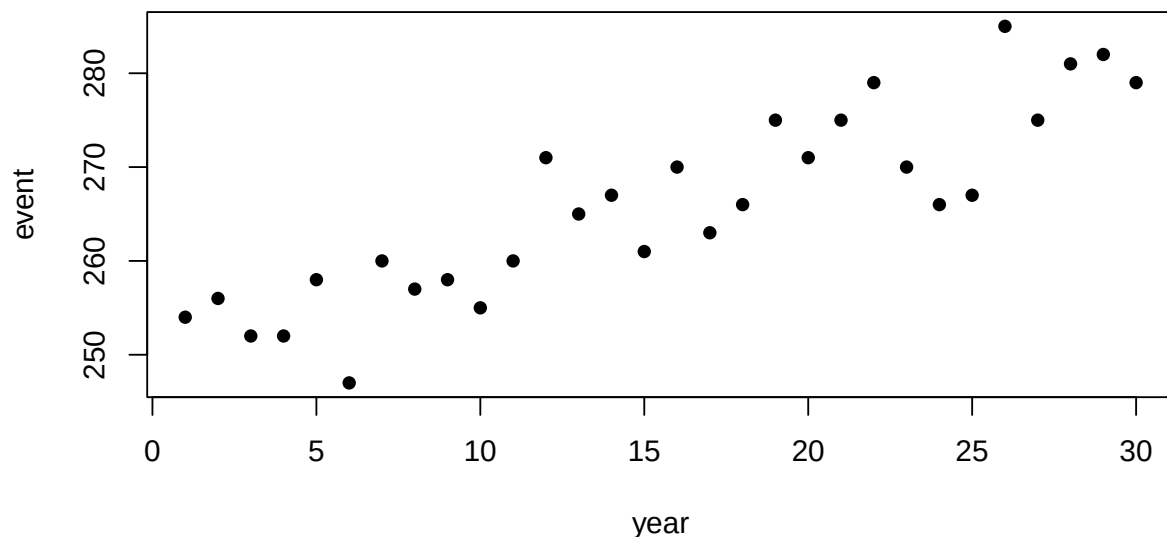
Ultimately, **phenometrics** is designed to be a broadly applicable collection of tools which calculate the point at which a time series' signal exceeds its noise (or in the case of ToM, when a time series' signal exceeds both its own noise and the signal of a comparison time series). However, it is important that end-users understand exactly what **phenometrics** is (and is not), and how they can modify the analyses it performs both within and without the package functions to tailor the tools within to their own study systems and research questions.

2.1 Directionality

The directionality of the tests in **phenometrics** is controlled by the **alt** argument(s) within each function. By default, the package functions are set to look for advancing (i.e. earlier) phenological trends, increasing environmental variables (presuming increasing temperature), and mismatch in either direction. If these directions are not the direction of interest for the end-user, the **alt** argument should be changed to match. For example, take the case of a species delaying its autumn phenology:

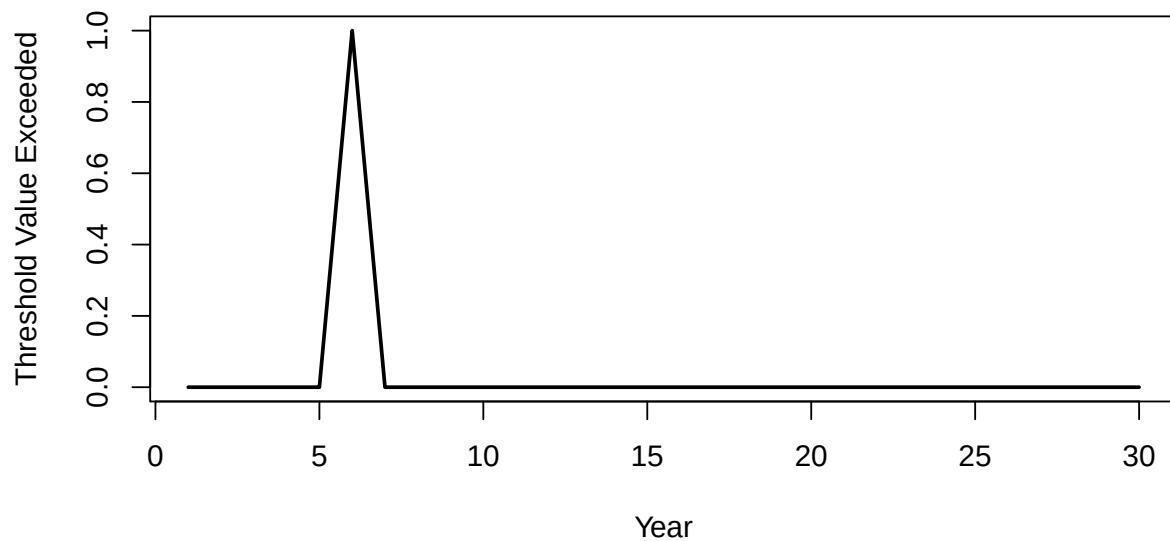
```
# Set seed
set.seed(12345)

# Create test dataset
year = seq(1,30,1)
event = round(rnorm(30, 250, 5)) + seq(1,30,1)
dataset = data.frame(year, event)
plot(data = dataset, event ~ year, pch = 16)
```



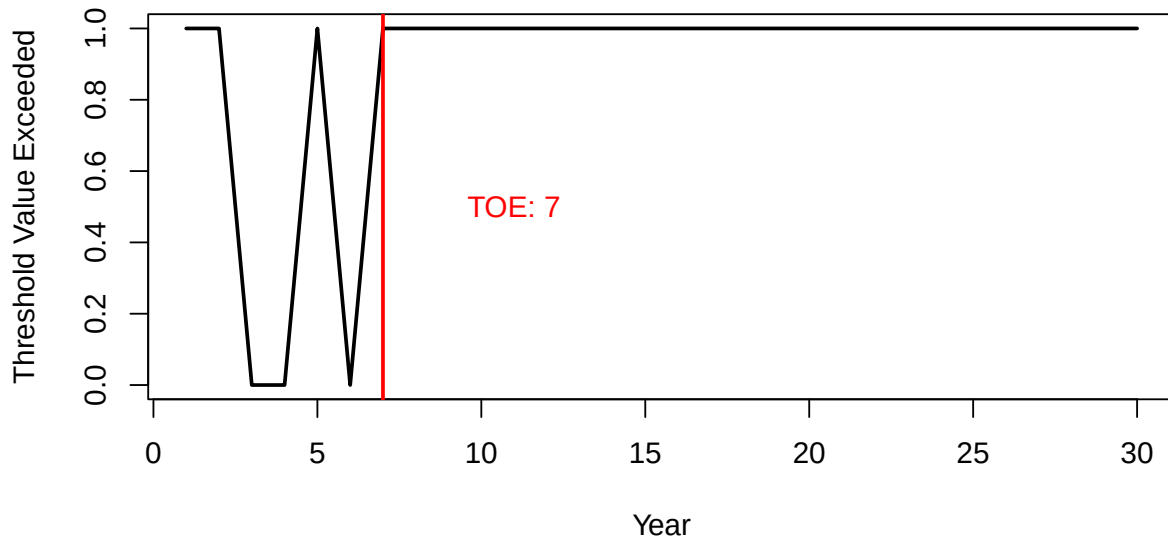
```
# Calculate empirical time of phenological emergence (ToPE)
test = emp_tope(dataset)
```

```
#> Warning in min(which(emerged == 1)): no non-missing arguments to min; returning
#> Inf
#> Warning in min(which(emerged == 1)): no non-missing arguments to min; returning
#> Inf
#> Warning in min(which(emerged == 1)): no non-missing arguments to min; returning
#> Inf
```

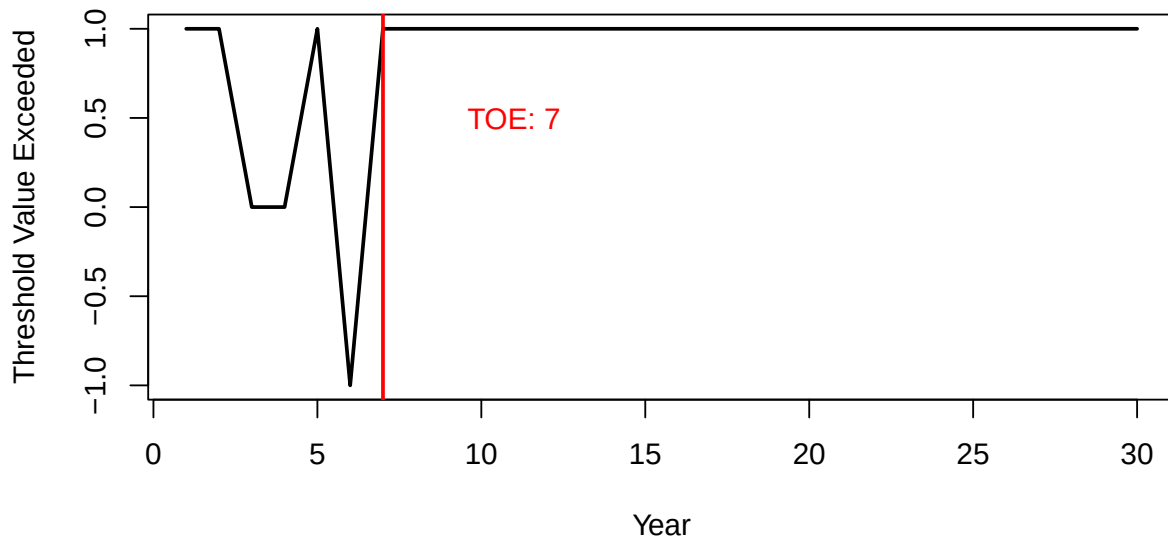


Emergence is not detected (resulting in the above warnings) because `emp_tope` defaults to looking for declining trends. This can be fixed by setting `alt = "less"` or `alt = "two.sided"`.

```
# Calculate empirical time of phenological emergence (ToPE)
less = emp_tope(dataset, alt = 'less')
```



```
twoside = emp_tope(dataset, alt = 'two.sided')
```



To prevent two-sided tests from emerging by exceeding alternating quantiles, two-sided empirical test results return 1 for a positive test where the original time series rises above the upper test quantile and -1 for positive tests where it falls below the lower test quantile. Emergence is assigned only when one of these two results occurs for `emt` consecutive years. This behaviour is unique to empirical testing, as the year-to-year nature of the statistical test makes it impossible for positive test results to switch directions. End users

should also note that changing the statistical test to `two.sided` feeds into the `alternative` argument of `ks.test`, which can cause the exact p-values to slightly shift compared to a one-sided test. This may change emergence timing in rare cases. This behaviour is unique to statistical testing.

End-users should be aware that aside from 1 and -1 in the empirical ToE functions, **phenometrics functions do not distinguish between increasing and decreasing time series when `alt = "two.sided"`, which may cause species with opposite trends to be combined in community-level functions.** We generally recommend one-sided tests on ToPE and ToEE for this reason. Running two one-sided tests may be preferable to running one two-sided test. This allows end users to distinguish undershifting responses (i.e. phenology is shifting, but not enough to offset change in the environment) from overshifting responses (i.e. phenology is shifting and environment at phenology has overshoot overall change in the environment). Users may identify overshifting by running one-sided tests in the expected direction for ToPE, and opposite of the expected direction for ToEE (e.g. checking for decreasing temperature at phenology. Note that using the community functions, this will be incorrectly labelled as undershifting).

2.2 Threshold Values

Both statistical and empirical ToE tests rely on various threshold values to quantify emergence. Generally, we do not recommend changing the values of these thresholds from their defaults unless there is clear and convincing reason to do so (i.e. ToE clearly does not match visual inspection of the time series).

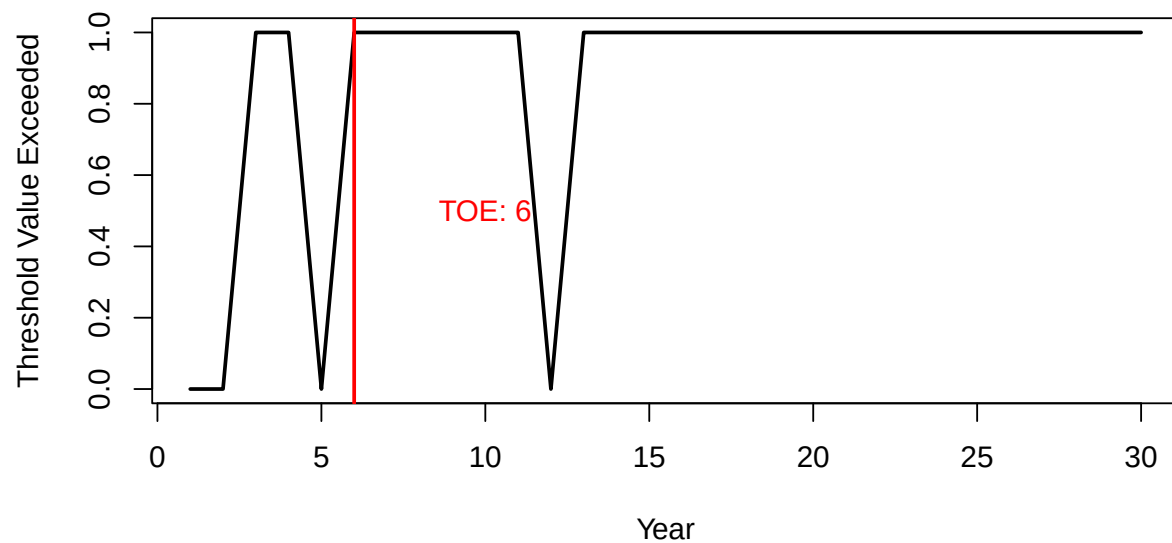
2.2.1 Empirical Thresholds

Empirical ToE test sensitivity can be tuned using the `quants` parameter. By default, it is set to the 25% and 75% quantiles, meaning a positive test result occurs when the original time series drops below the 25% quantile or rises above the 75% quantile of the detrended counterfactual (conditional on the value of `alt`). Moving these values closer to 0 and 1 respectively requires a greater difference between the original time series and the counterfactual for a positive test result to occur, making the test more conservative (and vice versa).

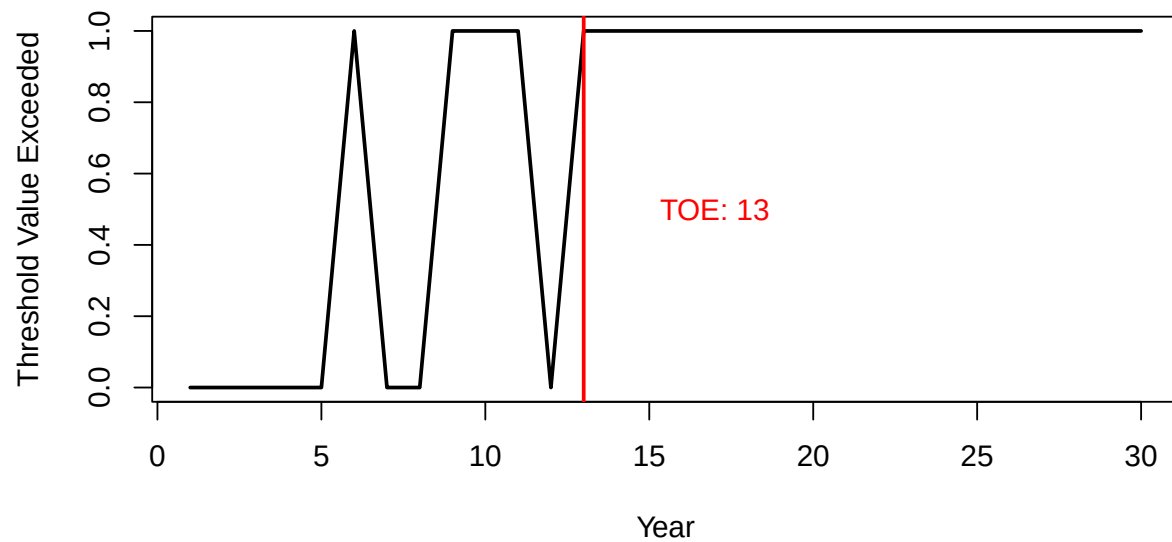
```
# Set seed
set.seed(12345)

# Create test dataset
year = seq(1,30,1)
event = round(rnorm(30, 100, 5))- seq(1,30,1)
dataset = data.frame(year, event)

# Calculate empirical time of phenological emergence (ToPE)
default = emp_tope(dataset)
```



```
edited = emp_tope(dataset, quants = c(0.025, 0.975))
```



2.2.2 Statistical Thresholds

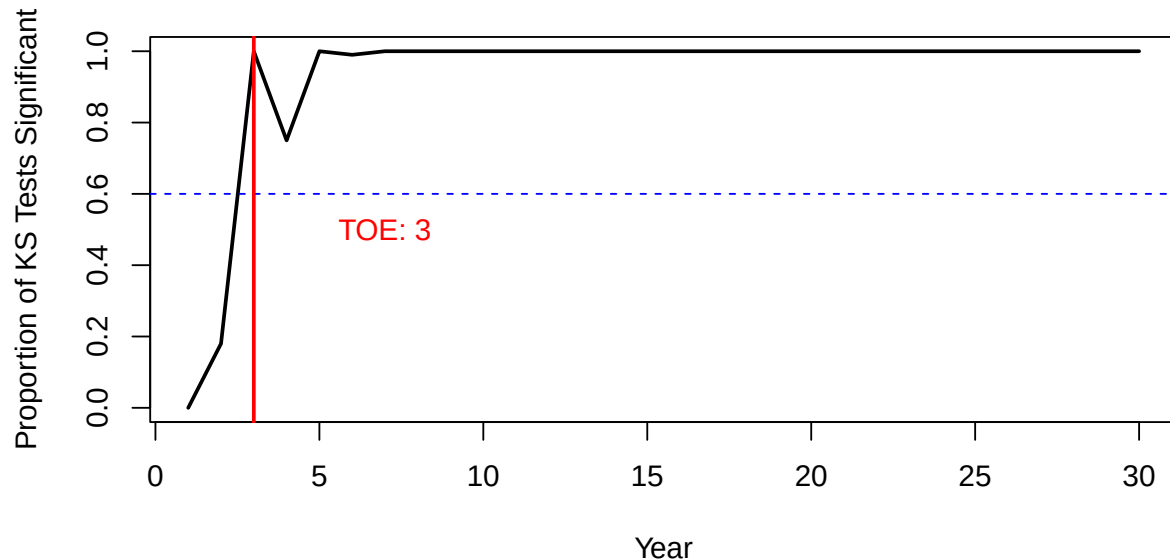
Statistical ToE test sensitivity is conditional on multiple parameters. `ks_t` is the proportion of bootstrapped KS tests which must be significant to return a positive test result, `alpha` is the significance test value of the

KS test, and `nboot` is the number of bootstrap KS tests performed. Increasing `nboot` reduces differences between run-to-run results in exchange for longer run time. Increasing `ks_t` closer to 1 and/or reducing `alpha` closer to 0 makes the test more conservative (and vice versa), though it should be noted that `alpha` = 0.05 is standard for statistical significance.

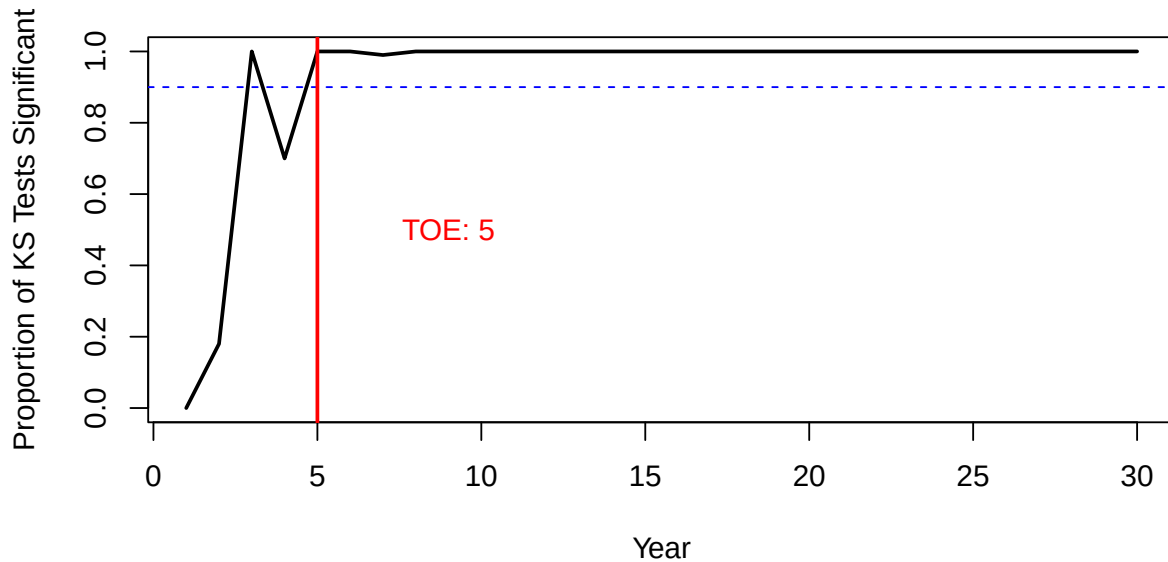
```
# Set seed
set.seed(12345)

# Create test dataset
year = rep(seq(1,30,1),30)
event = round(rnorm(900, 100, 5)) - year
dataset = data.frame(year, event) %>% arrange(year)

# Calculate empirical time of phenological emergence (ToPE)
default = ks_tope(dataset)
```



```
edited = ks_tope(dataset, ks_t = 0.9)
```



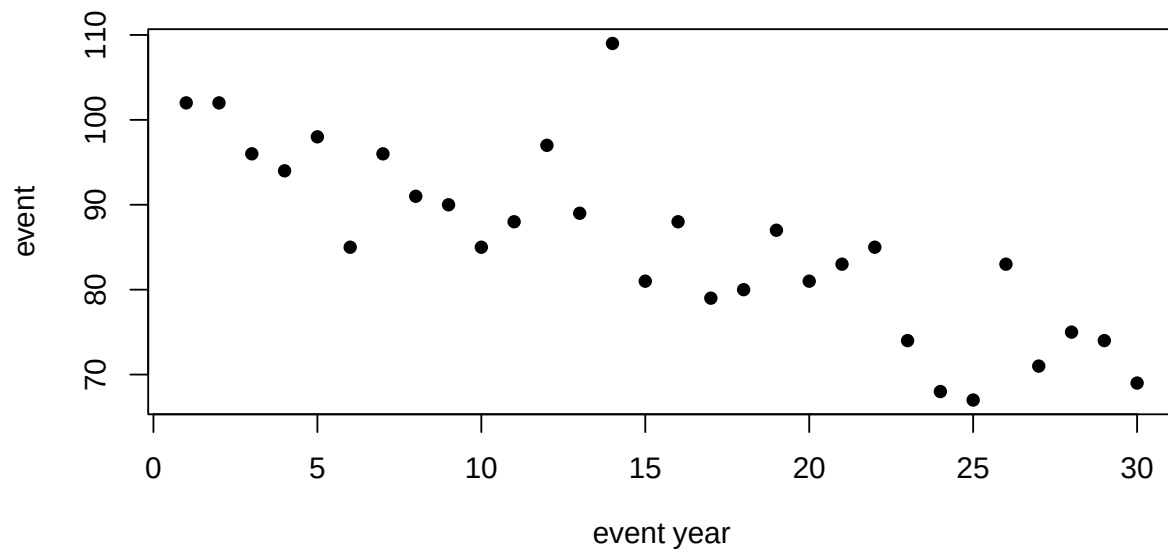
2.2.3 Emergence Threshold

The emergence threshold, controlled by `emt`, is the number of consecutive positive test results required to define emergence. It is set to 5 years by default, which is typically used in ToE literature. Increasing this value may make emergence take longer to occur, and vice versa. Changing `emt` only changes emergence timing when one or more negative test results occur after a positive test result. Lower values of `emt` are more susceptible to defining emergence early if the time series regresses back towards its baseline value, and/or uncertainty increases. High values of `emt` are more susceptible to delayed detection of emergence due to stochastic events in the opposite direction of the trend (see the code below for an example).

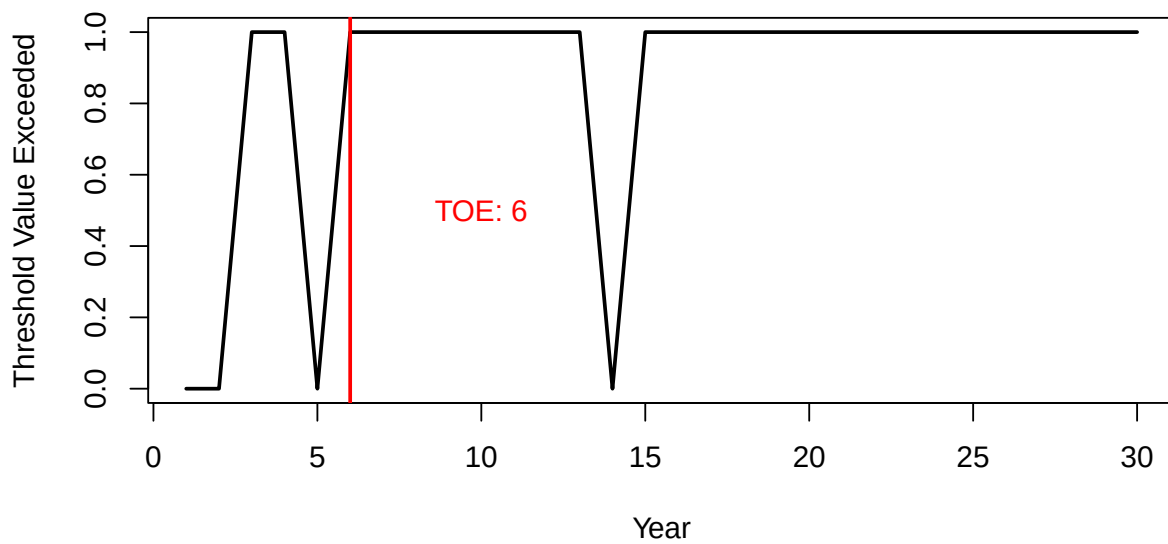
By default, all years after first emergence are considered emerged. This is to prevent emergence being negated by stochastic events after emergence has already occurred. This behaviour can be deactivated by setting `unemergence = T`, which may be useful in cases where a time series emerges, then reverts to its previous state.

```
# Set seed
set.seed(12345)

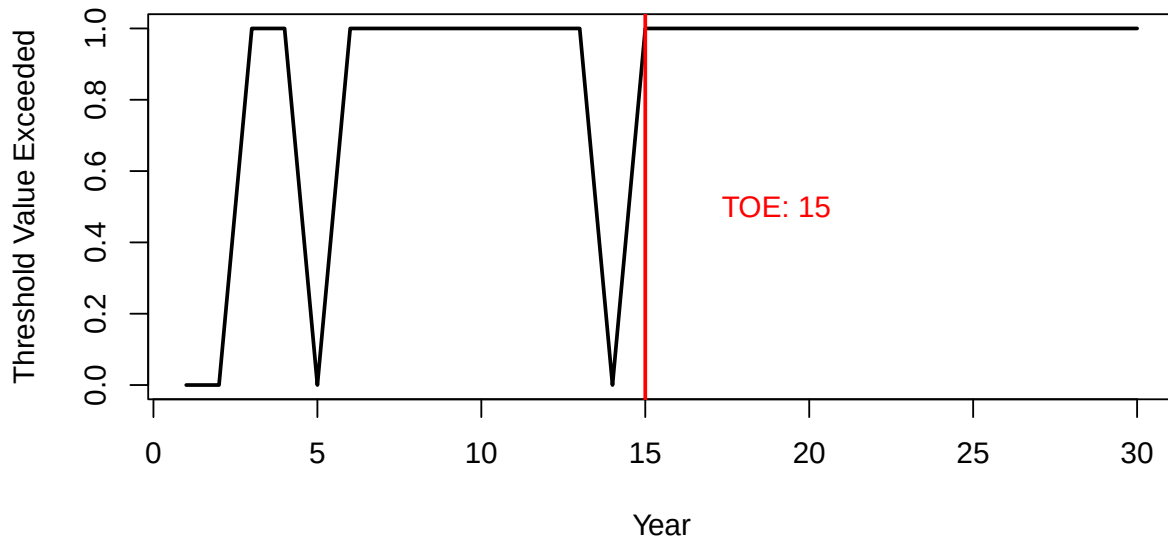
# Create test dataset
year = seq(1,30,1)
event = round(rnorm(30, 100, 5)) - seq(1,30,1); event[14] = event[14] + 20
dataset = data.frame(year, event)
plot(dataset, event ~ year, pch = 16)
```

```
# Calculate empirical time of phenological emergence (ToPE)
default = emp_tope(dataset)
```



```
edited = emp_tope(dataset, emt = 10)
```



2.3 Baseline Construction

One of the key features of *phenometrics* is that it creates a detrended counterfactual from the input time series. This serves as a baseline to test the original time series against for the purposes of quantifying emergence. This baseline construction methodology means that data doesn't need to be sacrificed to create a baseline to test against, and a stationary baseline can be generated even from time series which do not have any stationary period, both of which make this methodology preferable for short time series which have started well after the effects of climate change have begun.

The construction of this baseline is absolutely critical to the process of calculating ToE. Changes to the baseline can have significant effects on whether or when emergence is detected. We provide a few different options around baseline construction to allow users some latitude to customize the methodology to their own individual datasets, and to ensure that the constructed baseline is not misspecified. These include changing the model used to generate the baseline, and modifying which and how many years are used to calculate the baseline.

2.3.1 Curvilinear Detrending

By default, the detrended counterfactual is generated by fitting a linear model through the full time series, then subtracting its trend:

```
# Set seed
set.seed(12345)

# Create test dataset
year = seq(1,30,1)
event = round(rnorm(30, 100, 5)) - seq(1,30,1)
dataset = data.frame(year, event)
```

```

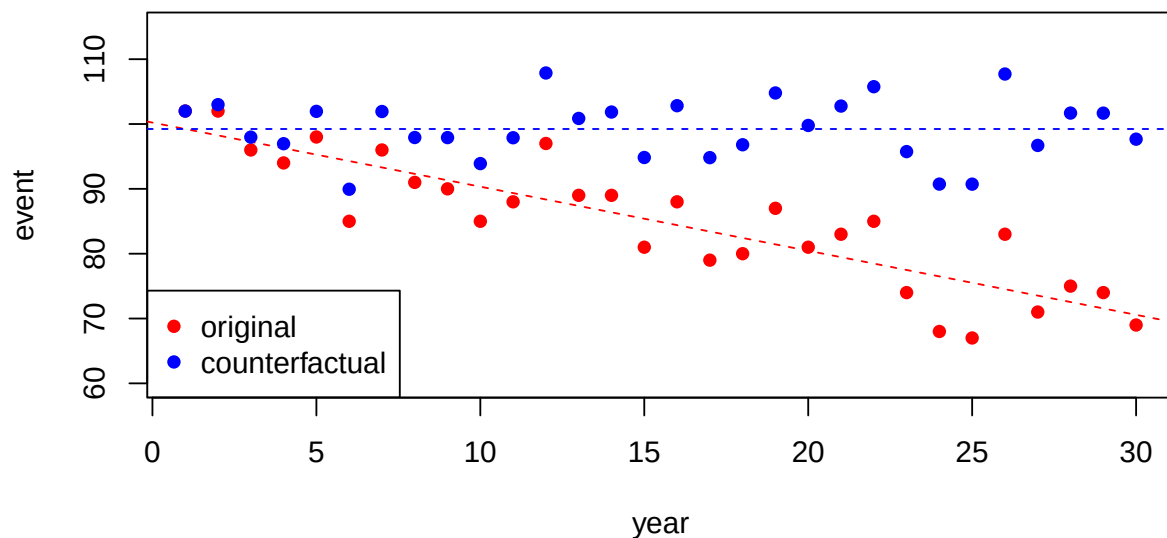
# Calculate linear model
event_lm = lm(data = dataset, event ~ year)

# Generate correction to calculate detrended counterfactual
correction = predict(event_lm, dataset) -
  predict(event_lm, dataset[dataset$year == min(dataset$year),])

# Calculate detrended counterfactual
counterfactual = dataset
counterfactual$event = dataset$event - correction

# Plot
plot(data = dataset, event ~ year, pch = 16, ylim = c(60,115), col = 'red')
abline(event_lm, lty = 'dashed', lwd = 1, col = 'red')
points(data = counterfactual, event ~ year, pch = 16, col = 'blue')
abline(lm(data = counterfactual, event ~ year), lty = 'dashed', lwd = 1, col = 'blue')
legend('bottomleft', col = c('red', 'blue'), legend = c('original', 'counterfactual'), pch = 16)

```



There are a few critical points about this base methodology that users should understand. The first is that by default, this method uses the entire length of the time series to create the counterfactual, meaning the counterfactual changes as data is added or removed. Secondly, because this detrending is performed with a linear model, **phenometrics** assumes by default that the trend in the time series is linear. If this assumption is incorrect the baseline will be poorly constructed, and emergence may not be defined properly. For example:

```

# Set seed
set.seed(12345)

# Create test dataset
dataset = NULL

```

```

year = seq(1,50,1)

# Create a test dataset with 10 replicates
for(i in 1:10){
  event = c(round(rnorm(30, 100, 5))- seq(1,30,1), rnorm(20,70,5))
  dataset_add = data.frame(year, event)
  dataset = rbind(dataset_add, dataset)
}

# Reorder by year
dataset = arrange(dataset, year)

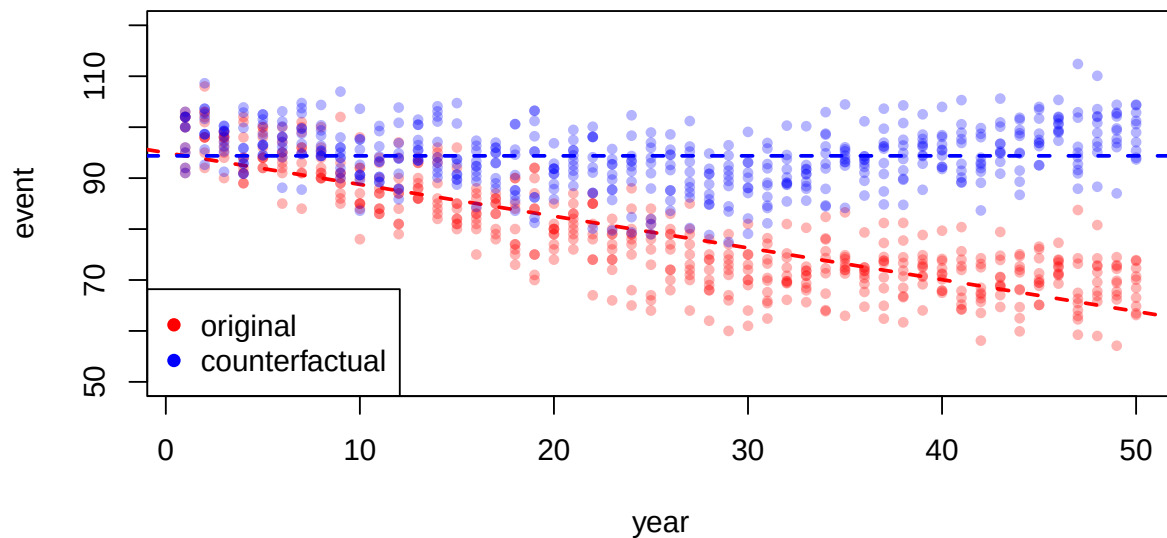
# Calculate linear model
event_lm = lm(data = dataset, event ~ year)

# Generate correction to calculate detrended counterfactual
correction = predict(event_lm, dataset) -
  predict(event_lm, dataset[dataset$year == min(dataset$year),])

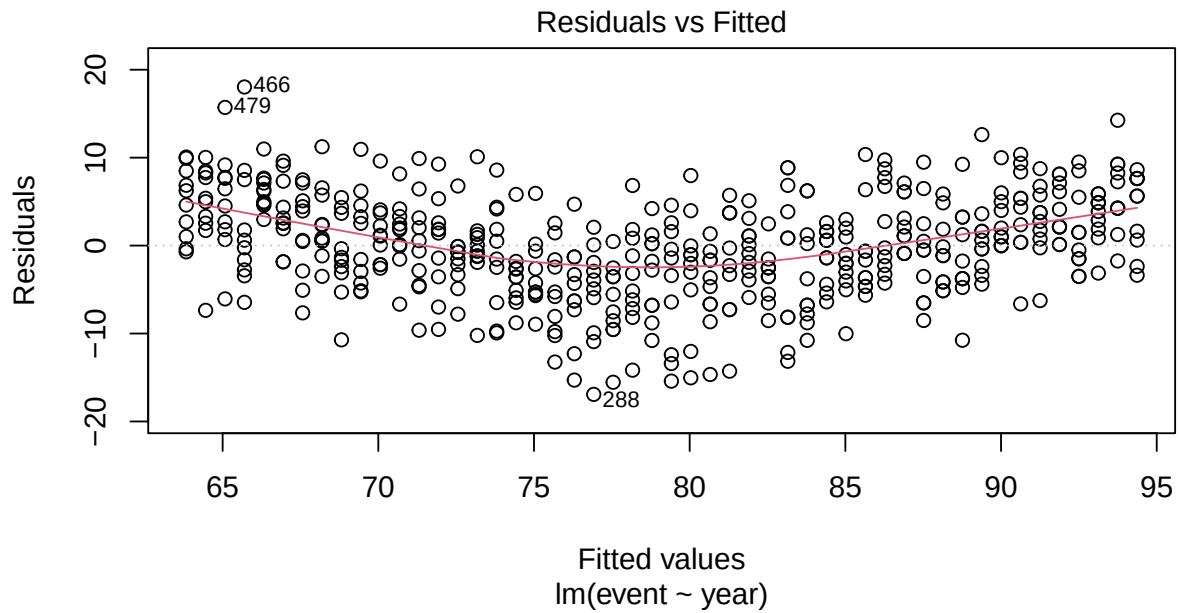
# Calculate detrended counterfactual
counterfactual = dataset
counterfactual$event = dataset$event-correction

# Plot
plot(data = dataset, event ~ year, pch = 16, ylim = c(50,120), col = alpha('red', 0.3), cex = 0.8)
abline(event_lm, lty = 'dashed', lwd = 2, col = 'red')
points(data = counterfactual, event ~ year, pch = 16, col = alpha('blue', 0.3), cex = 0.8)
abline(lm(data = counterfactual, event ~ year), lty = 'dashed', lwd = 2, col = 'blue')
legend('bottomleft', col = c('red', 'blue'),
      legend = c('original', 'counterfactual'), pch = 16)

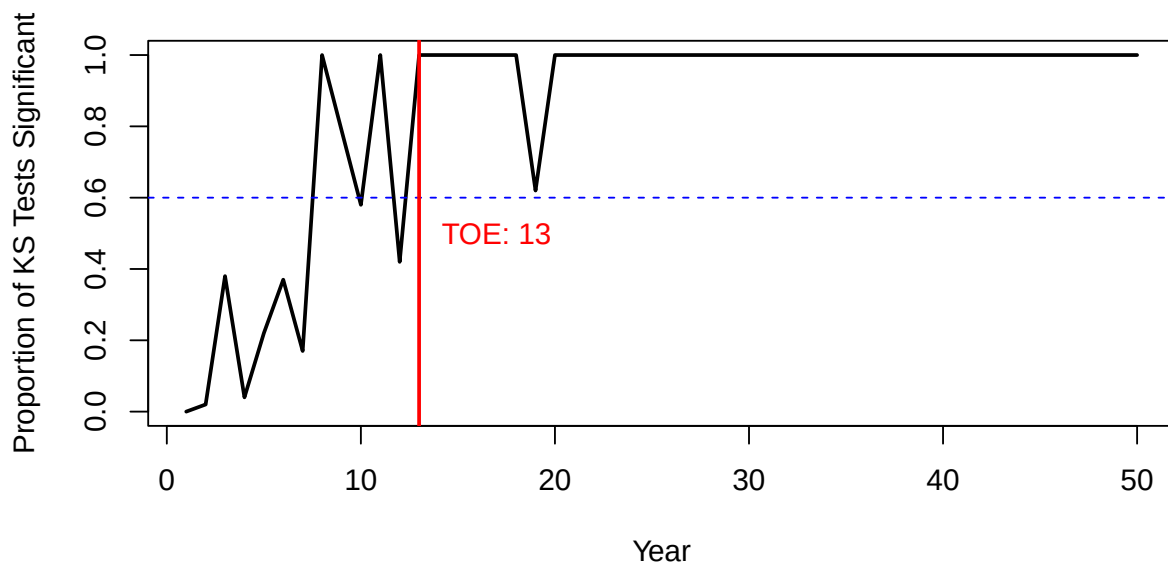
```



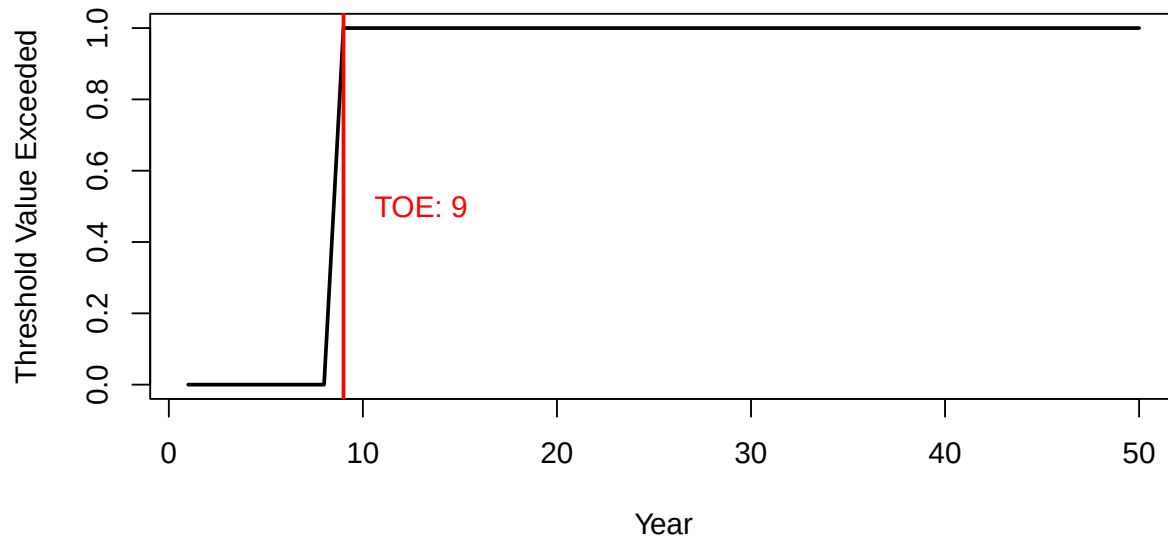
```
# Residuals vs fitted plot
plot(event_lm, which = 1)
```



```
# Run emergence
test = ks_tope(dataset)
```



```
test = emp_tope(dataset)
```



In this case, because the input data is not linear, the counterfactual is not flat throughout the time series. We recommend users check their data for linearity where it may be of concern using the residuals vs fitted plot, or similar. Results may be affected variably depending on the counterfactual pattern. Here, emergence takes longer to occur because the counterfactual is closer to the original time series than it should be through the early part of the time series. However, this may not always be the case:

```
# Set seed
set.seed(12345)

# Create test dataset
dataset = NULL
year = seq(1,50,1)

# Create a test dataset with 10 replicates
for(i in 1:10){
  event = c(round(rnorm(20, 100, 5)), rnorm(30,100,5)- seq(1,30,1))
  dataset_add = data.frame(year, event)
  dataset = rbind(dataset_add, dataset)
}

# Reorder by year
dataset = arrange(dataset, year)

# Calculate linear model
event_lm = lm(data = dataset, event ~ year)

# Generate correction to calculate detrended counterfactual
correction = predict(event_lm, dataset) -
```

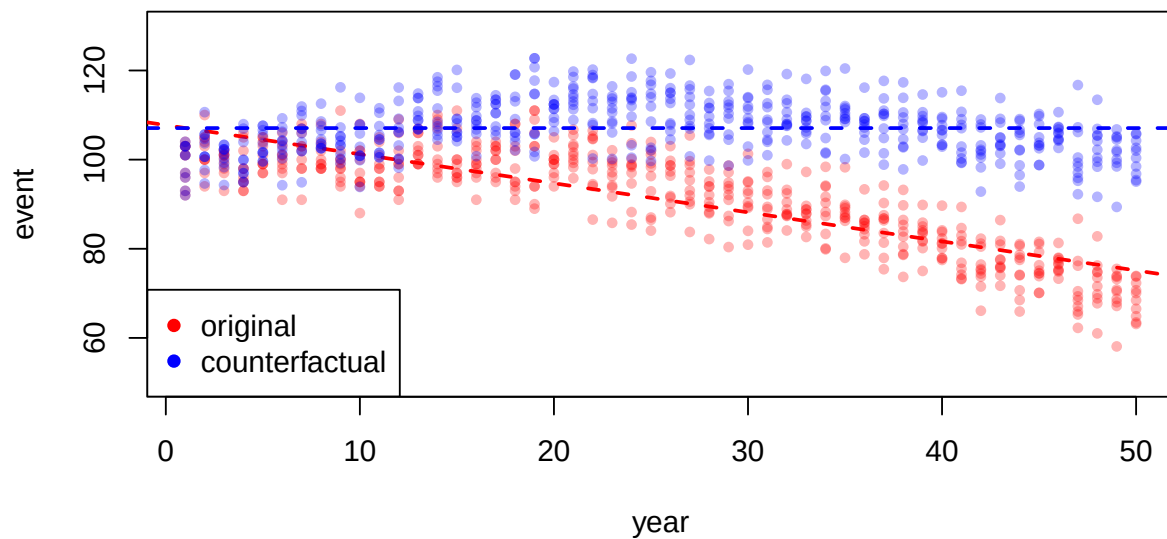
```

predict(event_lm, dataset[dataset$year == min(dataset$year),])

# Calculate detrended counterfactual
counterfactual = dataset
counterfactual$event = dataset$event-correction

# Plot
plot(data = dataset, event ~ year, pch = 16, ylim = c(50,130), col = alpha('red', 0.3), cex = 0.8)
abline(event_lm, lty = 'dashed', lwd = 2, col = 'red')
points(data = counterfactual, event ~ year, pch = 16, col = alpha('blue', 0.3), cex = 0.8)
abline(lm(data = counterfactual, event ~ year), lty = 'dashed', lwd = 2, col = 'blue')
legend('bottomleft', col = c('red', 'blue'), legend = c('original', 'counterfactual'), pch = 16)

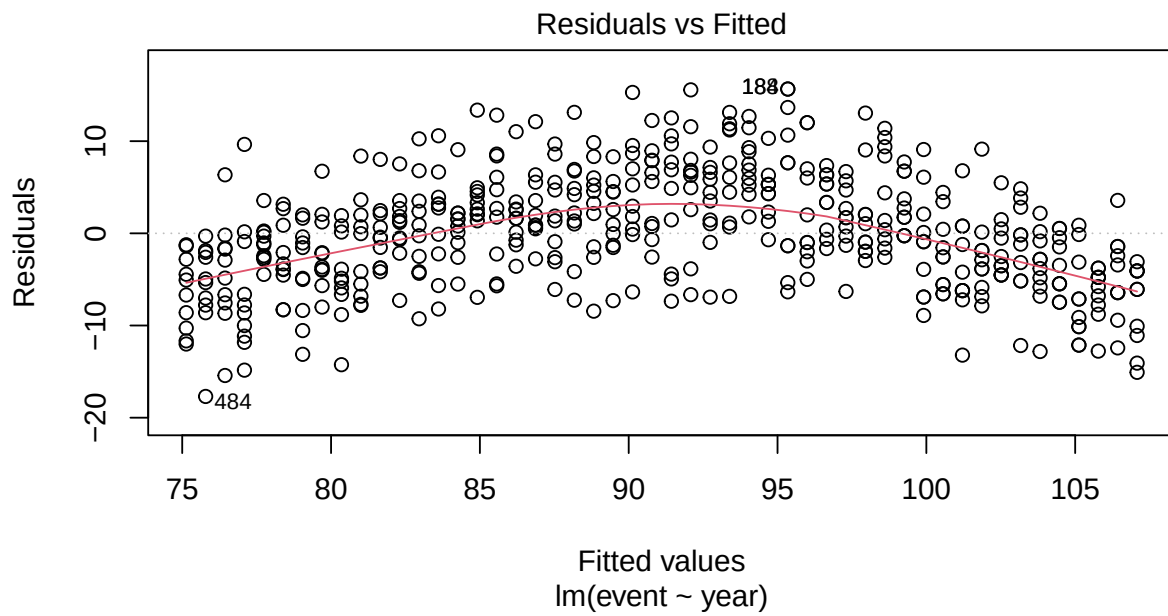
```



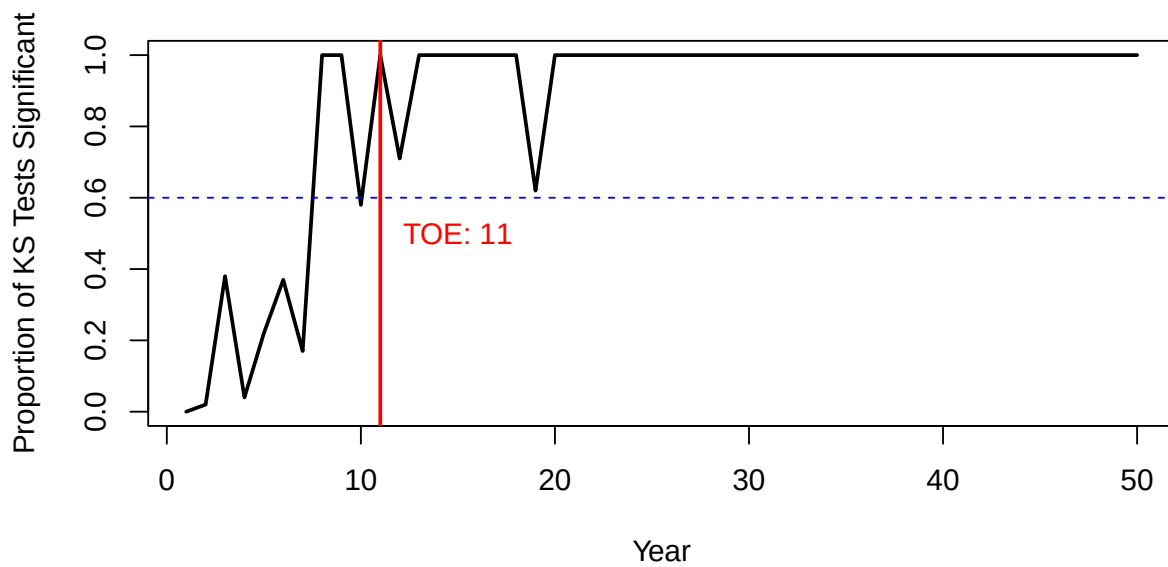
```

# Residuals vs fitted plot
plot(event_lm, which = 1)

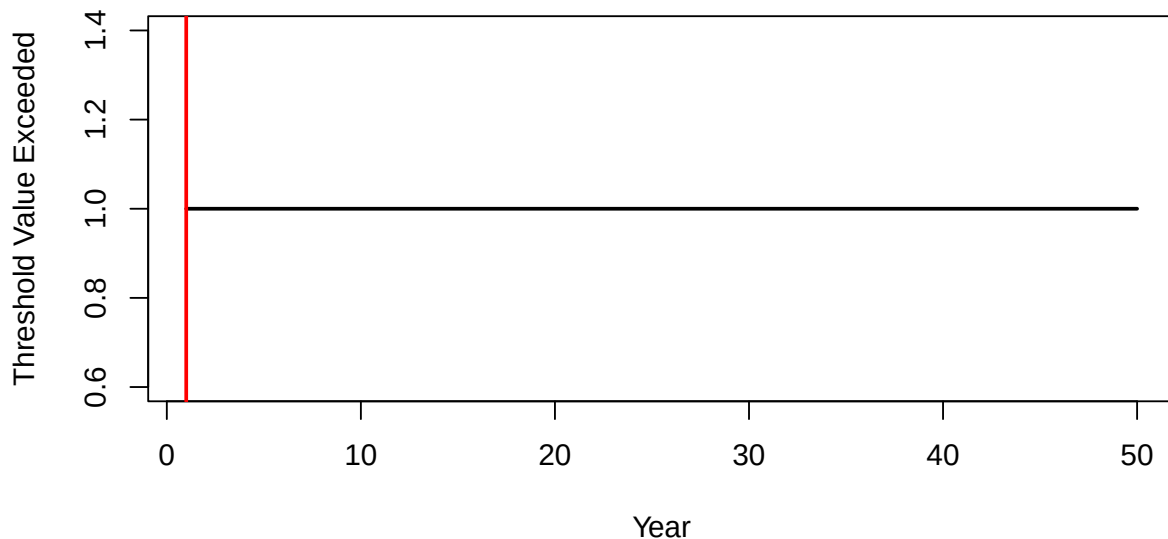
```



```
# Run emergence
test = ks_tope(dataset)
```



```
test = emp_tope(dataset)
```

In this case, statistical emergence is detected within the first 20 years, despite the original time series being flat over that period. This is because the misspecification of the model caused the counterfactual to increase over the early part of the time series, causing a difference to be detected between the two despite no change in the original data. Meanwhile, empirical emergence is detected immediately because the pattern in the counterfactual has artificially increased the test quantiles to the point that the beginning of the time series is below the lower quantile.

As shown above, *phenometrics* performs detrending by fitting a model to the original time series, then subtracting the trend of that model. Hypothetically, this methodology could be performed using any model. In addition to the simple linear model default, we allow for the addition of an x^2 term to the detrending model, making it curvilinear by setting `model = 'curvilinear'`. Curvilinear patterns are common in climate change-related data, and are much better at handling plateaus and other slope changes in the data such as the examples shown previously:

```
# Set seed
set.seed(12345)

# Create test dataset
dataset = NULL
year = seq(1,50,1)

# Create a test dataset with 10 replicates
for(i in 1:10){
  event = c(round(rnorm(20, 100, 5)), rnorm(30,100,5)- seq(1,30,1))
  dataset_add = data.frame(year, event)
  dataset = rbind(dataset_add, dataset)
}

# Reorder by year
dataset = arrange(dataset, year)

# Calculate linear model
```

```

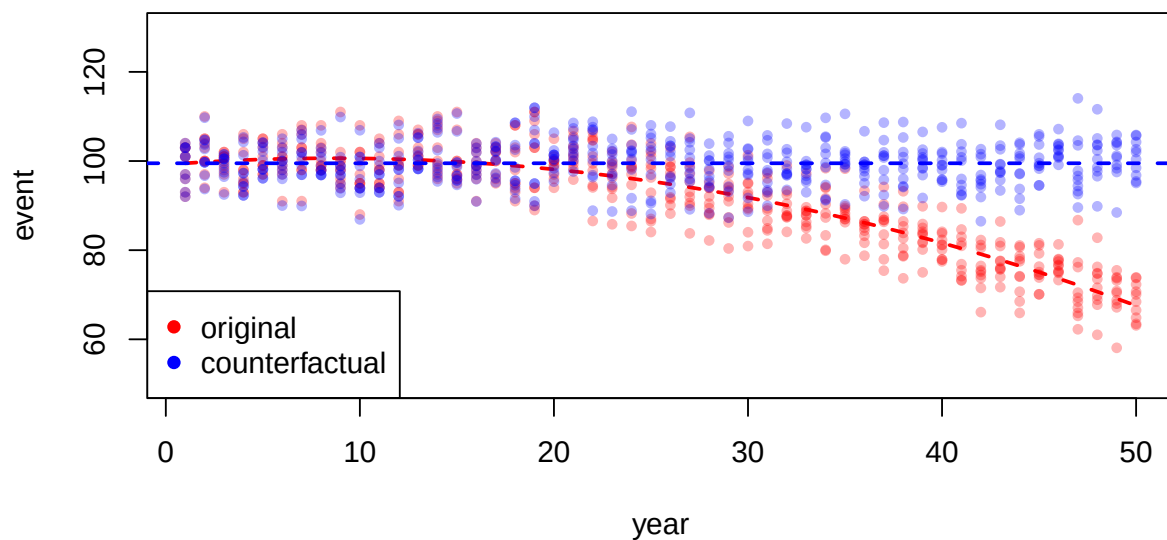
event_lm = lm(data = dataset, event ~ year + I(year^2))
preds = predict(event_lm, newdata = list(year = year))

# Generate correction to calculate detrended counterfactual
correction = predict(event_lm, dataset) -
  predict(event_lm, dataset[dataset$year == min(dataset$year),])

# Calculate detrended counterfactual
counterfactual = dataset
counterfactual$event = dataset$event - correction

# Plot
plot(data = dataset, event ~ year, pch = 16, ylim = c(50,130), col = alpha('red', 0.3), cex = 0.8)
lines(preds ~ year, col = 'red', lty = 'dashed', lwd = 2)
points(data = counterfactual, event ~ year, pch = 16, col = alpha('blue', 0.3), cex = 0.8)
abline(lm(data = counterfactual, event ~ year), col = 'blue', lty = 'dashed', lwd = 2)
legend('bottomleft', col = c('red', 'blue'), legend = c('original', 'counterfactual'), pch = 16)

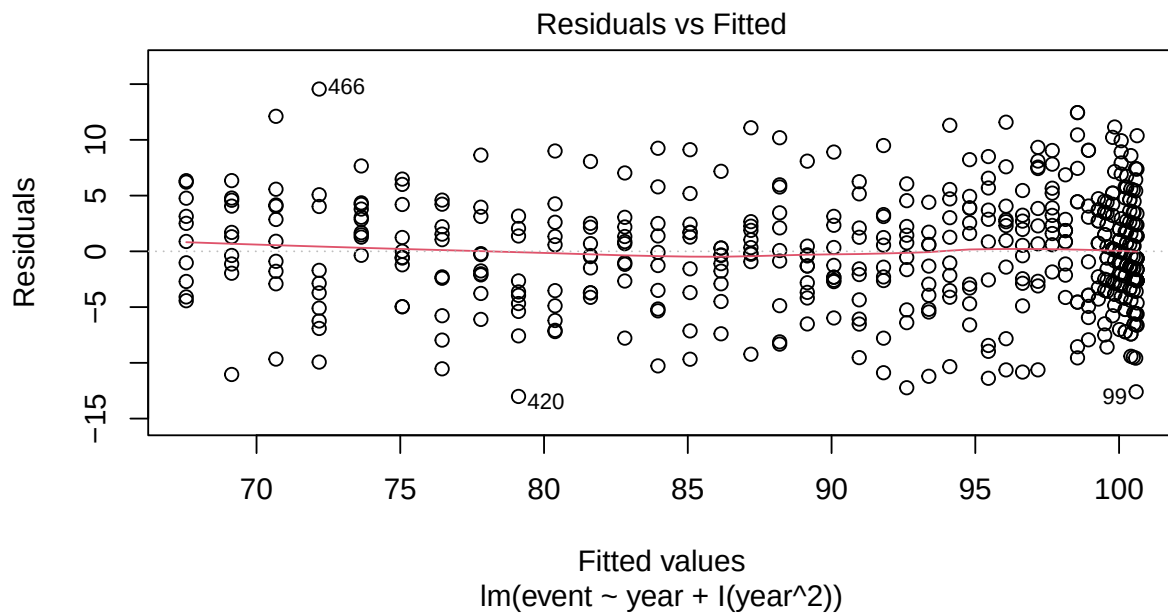
```



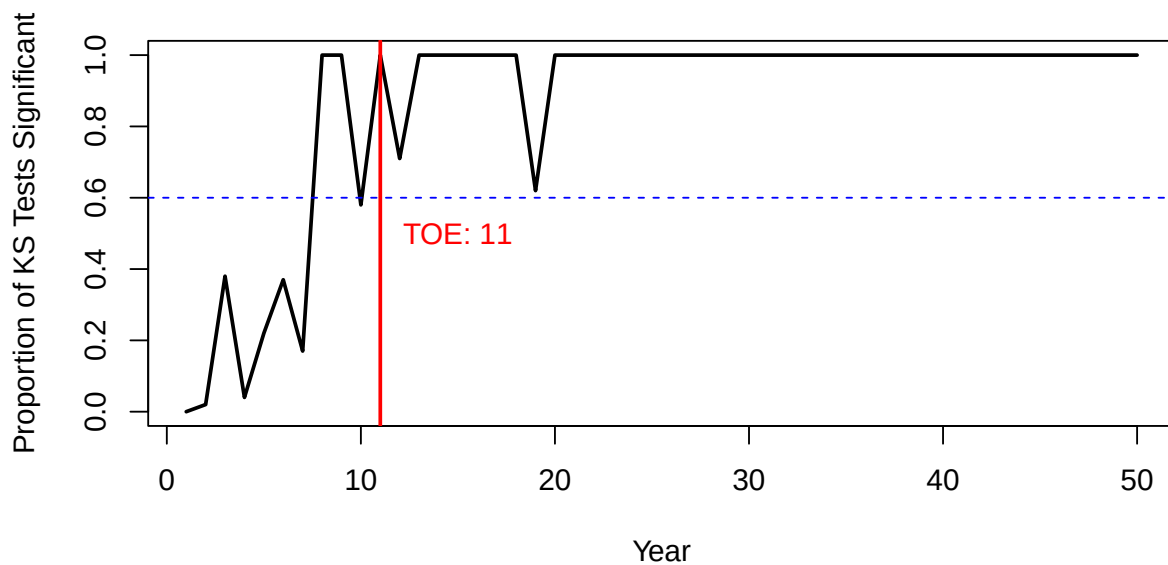
```

# Residuals vs fitted plot
plot(event_lm, which = 1)

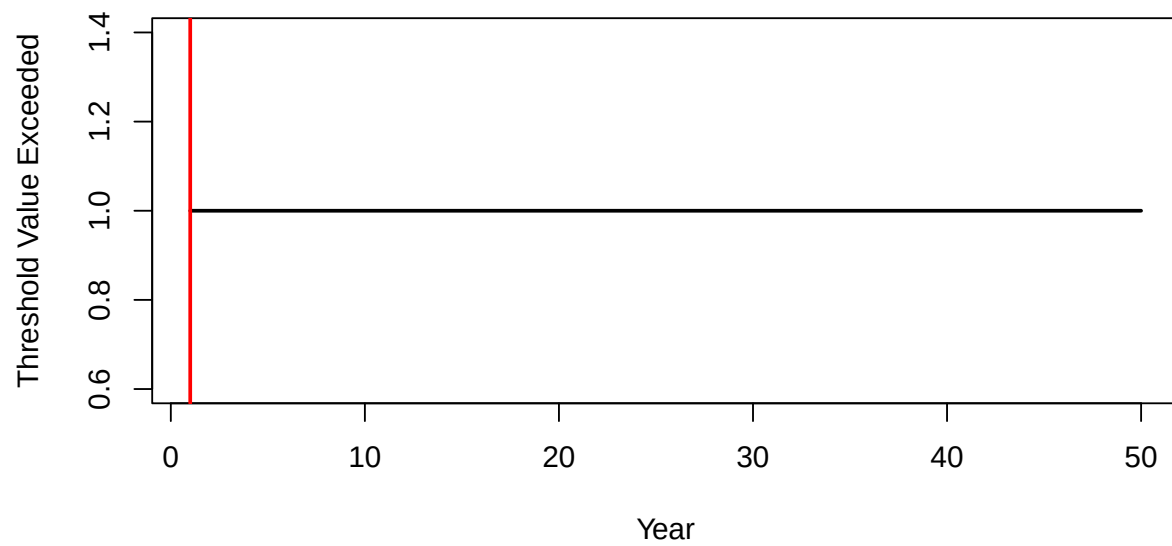
```



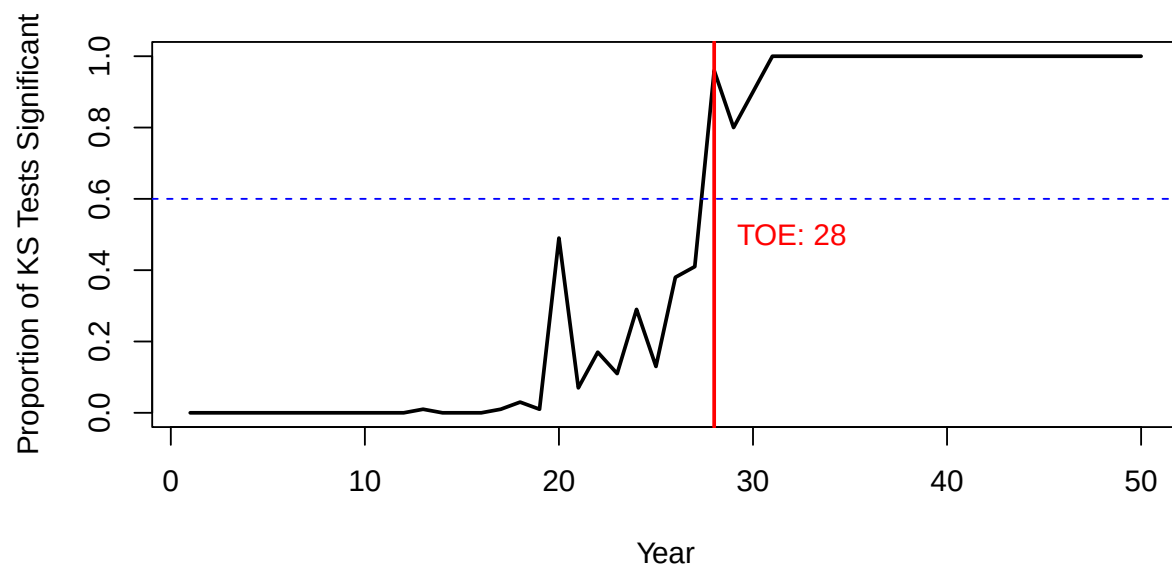
```
# Run emergence testing
test = ks_tope(dataset, model = 'linear')
```



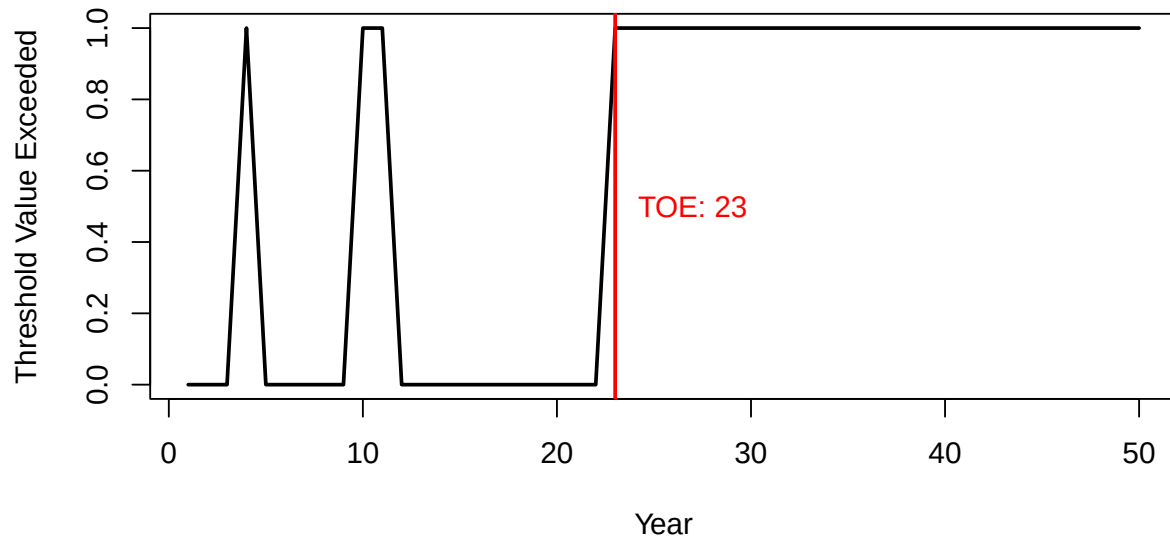
```
test = emp_tope(dataset, model = 'linear')
```



```
test = ks_tope(dataset, model = 'curvilinear')
```



```
test = emp_tope(dataset, model = 'curvilinear')
```



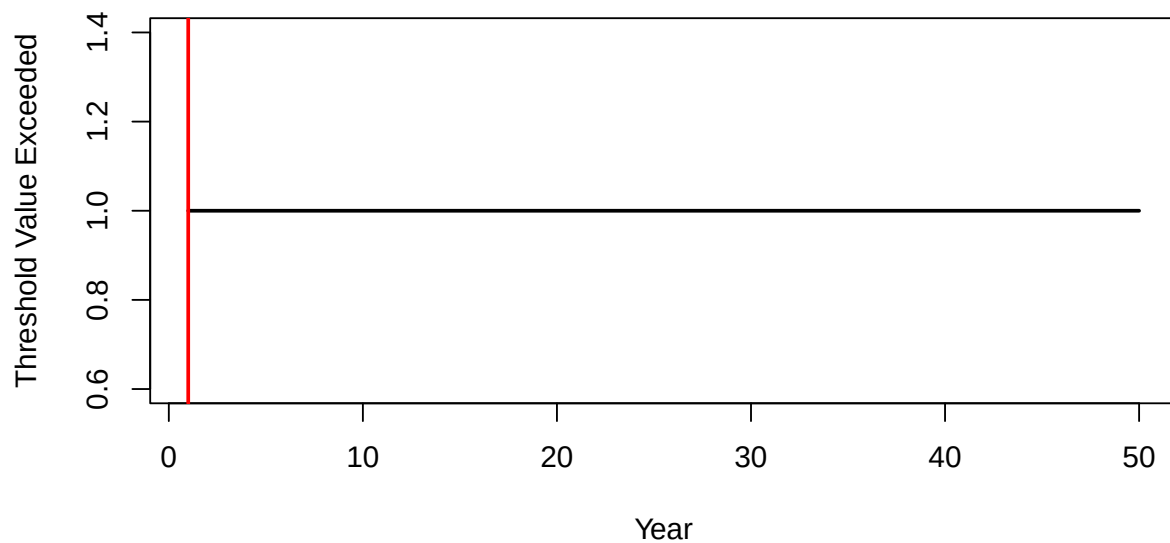
Here, the counterfactual is properly flat around the baseline value for the full time series, and is not closer to the original time series than it should be at any point in the time series.

2.3.2 Limited Baseline

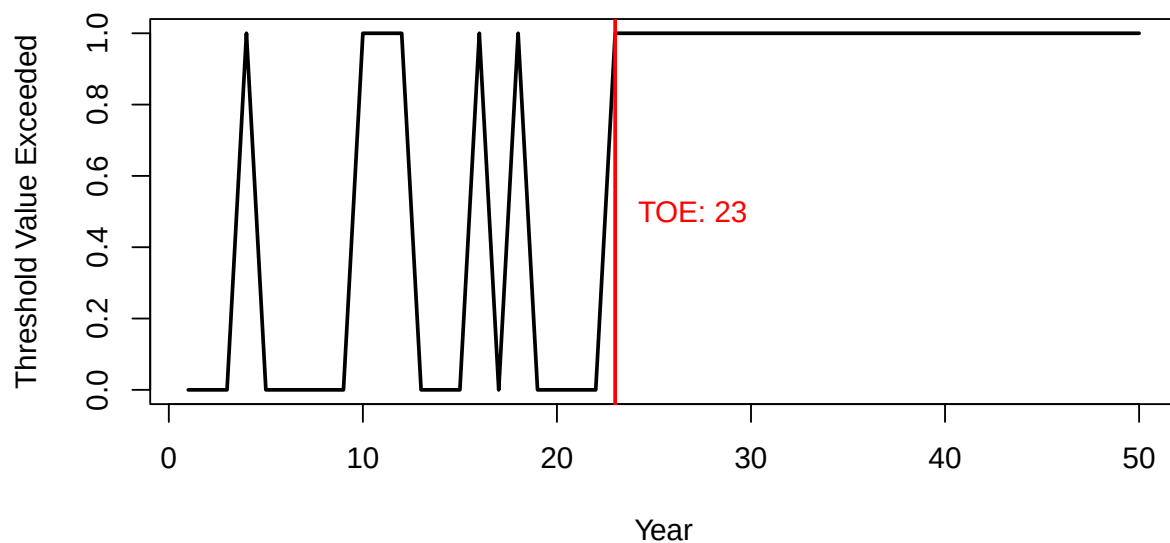
One important difference between the statistical and empirical ToE methodologies is that the statistical methodology tests the original time series against the detrended counterfactual on an annual basis, while the empirical methodology tests each year of the original time series against test quantiles generated from the entire detrended counterfactual. As a result, the empirical method is more sensitive to time series length and variance patterns than the statistical method. This can cause changes in the exact year of emergence of a time series when data is added or removed from the end of the time series, even after emergence has already occurred. In some cases this behaviour may be desirable - for example, if adding further years to the time series brings the counterfactual closer to a more representative estimate of the amount of year-to-year variability in a species' phenology. In others, it may be undesirable, such as if changing variability in a species' phenology is driven by climate change, or by changes in sampling protocol.

As a method of controlling for these changes, we have included functionality in the empirical ToE tests to limit the baseline period of the detrended counterfactual used to calculate the test quantiles. This is performed through a combination of the `max_y` and `baseline` arguments. `max_y` represents the maximum number of years used to calculate the baseline period (by default it is set to 0, which deactivates this functionality and uses the whole time series). Setting `baseline = 'static'` (the default) fixes the baseline period to the first `max_y` years of the time series (see next section for more details). This can also be used to handle nonlinear patterns in the data, so long as the data is linear across the baseline period used to calculate the test quantiles. For example, reusing the dataset from above:

```
# Run emergence testing
test = emp_tope(dataset)
```



```
test = emp_tope(dataset, max_y = 20, baseline = 'static')
```



In this example, the counterfactual is misspecified using the default settings because the linearity assumption is violated, leading to an expansion of the test quantiles, and no emergence being detected. However, limiting the baseline to the first 20 years over which the data is linear and the assumption is not violated allows us to detect emergence properly.

This functionality is not built into the statistical method because the statistical method performs year-to-year

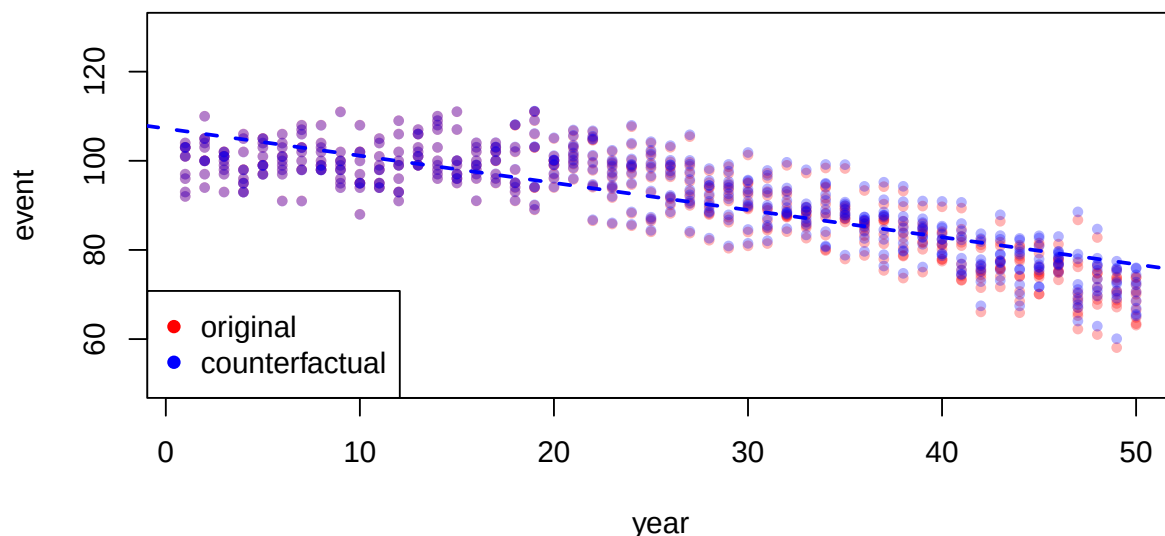
testing, which is incompatible with a true static limited baseline. If users are concerned about ToE values changing due to the addition of future data via small changes in the value of the trend, this can be prevented by passing the `subset` argument to `lm()` inside the statistical emergence functions. This argument may be implemented to limit which years are used to fit the linear models inside the function, and thus detrend the counterfactual. In this case, the detrended counterfactual is extrapolated in years beyond the `subset`, which may be problematic if the trend in the time series changes significantly over time. Again, reusing the same dataset:

```
# Calculate linear model
event_lm = lm(data = dataset, event ~ year + I(year^2), subset = which(dataset$year <= 20))
preds = predict(event_lm, newdata = list(year = year))

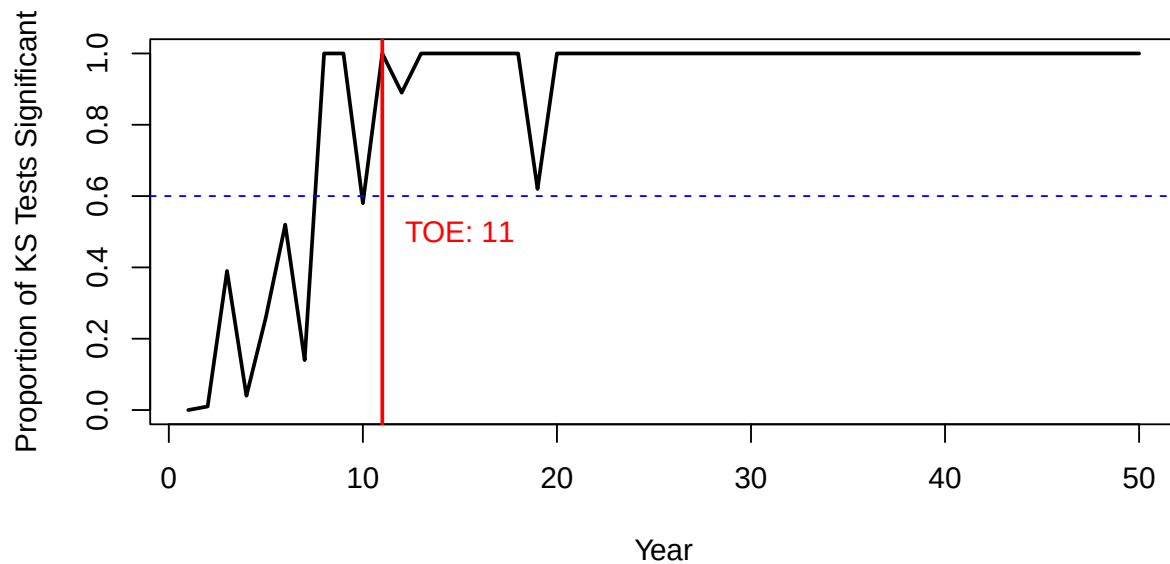
# Generate correction to calculate detrended counterfactual
correction = predict(event_lm, dataset) -
  predict(event_lm, dataset[dataset$year == min(dataset$year),])

# Calculate detrended counterfactual
counterfactual = dataset
counterfactual$event = dataset$event - correction

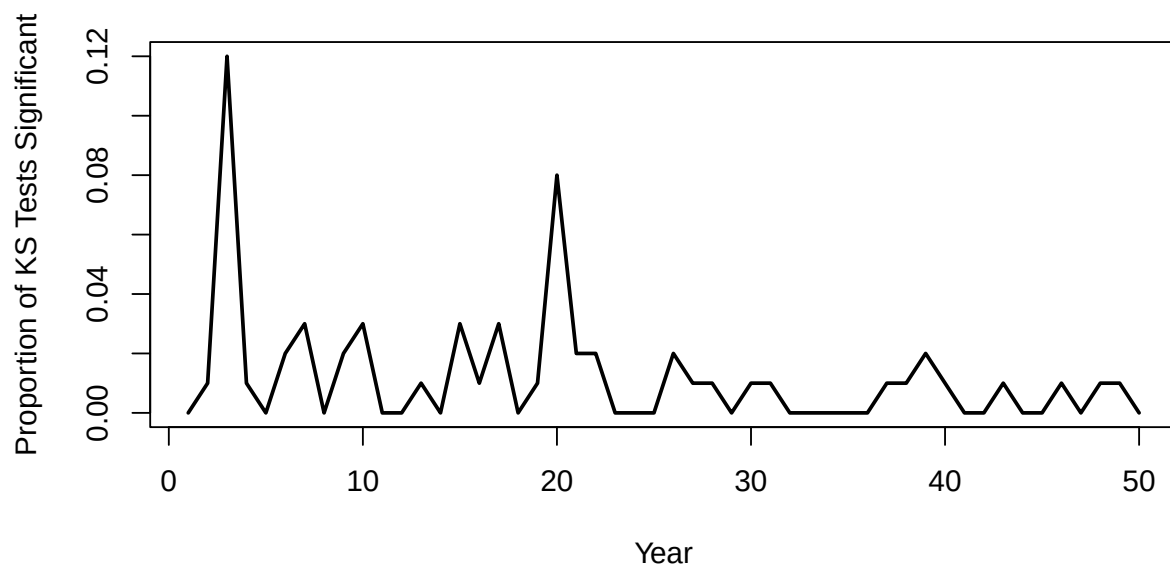
# Plot
plot(data = dataset, event ~ year, pch = 16, ylim = c(50,130), col = alpha('red', 0.3), cex = 0.8)
# lines(preds ~ year, col = 'red', lty = 'dashed', lwd = 2)
points(data = counterfactual, event ~ year, pch = 16, col = alpha('blue', 0.3), cex = 0.8)
abline(lm(data = counterfactual, event ~ year), col = 'blue', lty = 'dashed', lwd = 2)
legend('bottomleft', col = c('red', 'blue'), legend = c('original', 'counterfactual'), pch = 16)
```



```
# Run emergence testing
test = ks_tope(dataset)
```



```
test = ks_tope(dataset, subset = which(dataset$year <= 20))
#> Warning in min(which(emerged == 1)): no non-missing arguments to min; returning
#> Inf
#> Warning in min(which(emerged == 1)): no non-missing arguments to min; returning
#> Inf
#> Warning in min(which(emerged == 1)): no non-missing arguments to min; returning
#> Inf
```



Here, using `subset` this way is inadvisable because the trend at the start of the time series is not representative of the trend later in the time series, so the counterfactual is not flat. Ideally, `subset` should only be used in this way if emergence has already occurred within the subsetted years.

2.3.3 Moving Window

In some unique cases, users may have a very long time series which contains multiple patterns. For an extreme example, see the Kyoto peak cherry tree blossom time series. In such cases, users may be interested in these changing patterns - for example, if a species phenology emerges, and then settles into a new stable period. To handle cases such as these, we have built in functionality to apply our methodology across a moving window. As in the limited baseline case, the length of the moving window is determined by the `max_y` parameter, which represents the maximum number of years used in the moving window (by default it is set to 0, which deactivates this functionality and uses the whole time series). To enable the moving window approach for empirical tests, users must also set `baseline = 'moving'`. As there is no static limited baseline approach in the statistical methodology, setting `max_y` to any value greater than 0 will activate the moving window approach.

For example, if `max_y = 20` (and `baseline = 'moving'` if using an empirical test) the methodology will calculate emergence for the first 20 years of the time series as if only given the first 20 years. Year 21 will be calculated as if given years 2 to 21, year 22 as if given years 3 to 22, and so on to the end of the time series.

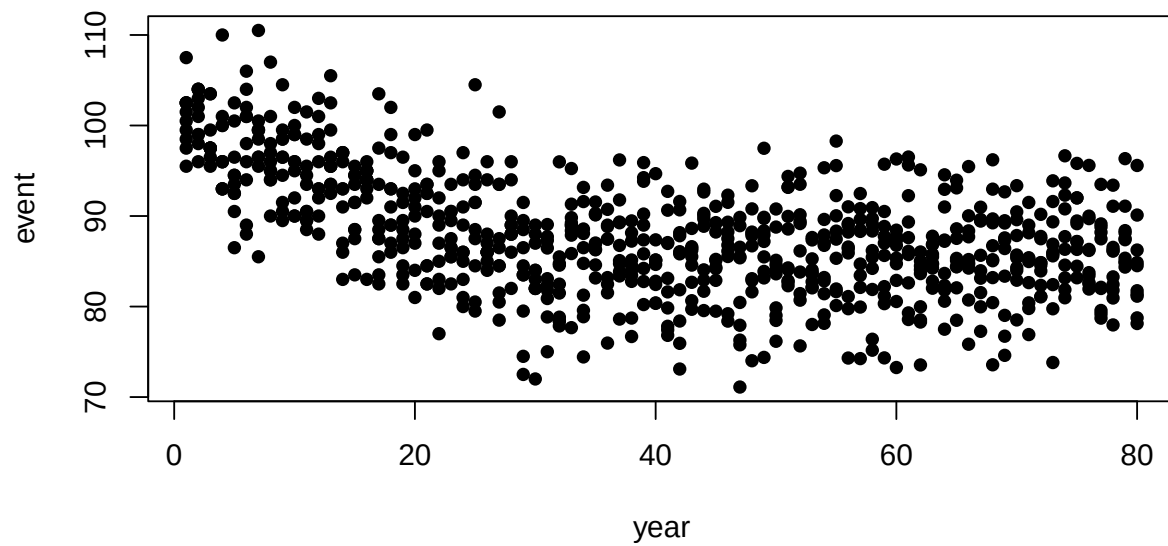
In this scenario, the default statistical test is misspecified because the assumption of linearity is violated, leading to a poorly estimated ToE value which occurs while the time series is actually static:

```
# Set seed
set.seed(12345)

# Create test dataset
dataset = NULL
year = seq(1,80,1)

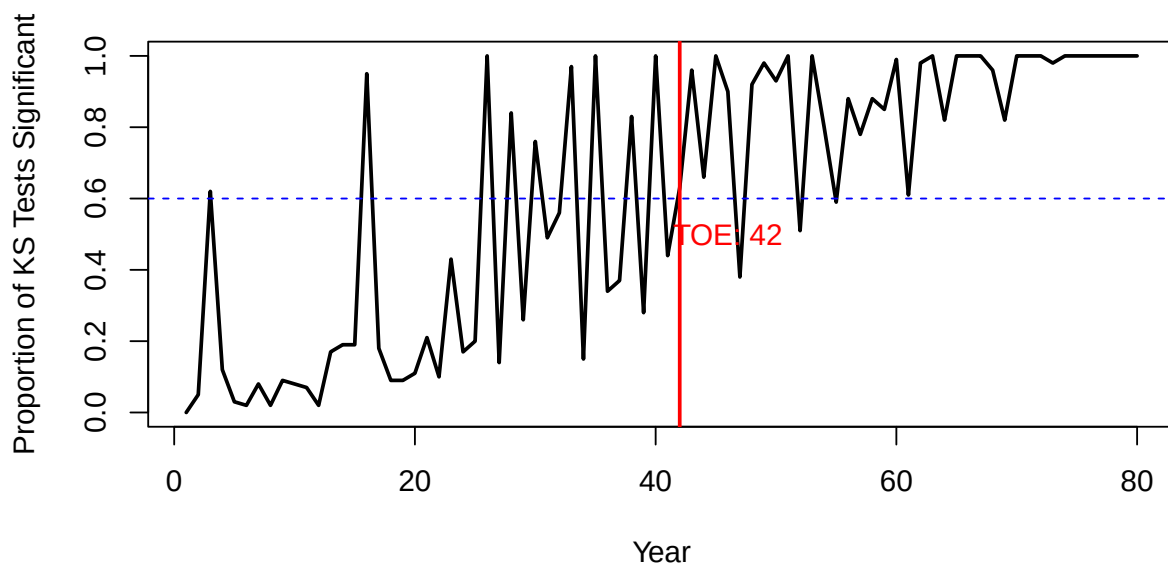
# Create a test dataset with 10 replicates
for(i in 1:10){
  event = c(round(rnorm(30, 100, 5))- seq(0.5,15,0.5), rnorm(50,85,5))
  dataset_add = data.frame(year, event)
  dataset = rbind(dataset_add, dataset)
}

# Plot data
plot(data = dataset, event ~ year, pch = 16)
```



```
# Reorder by year
dataset = arrange(dataset, year)

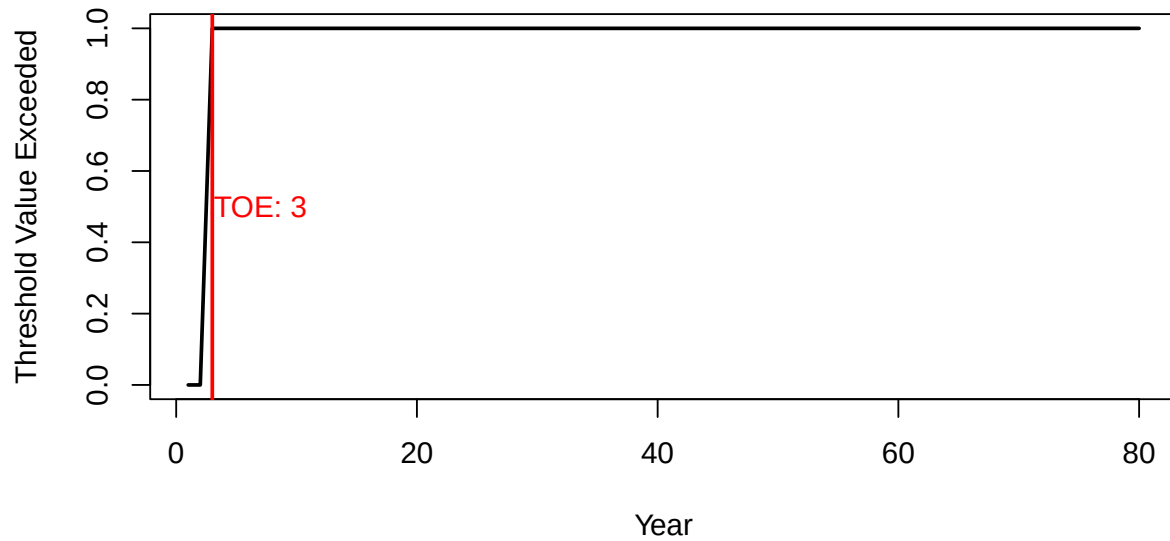
# Run emergence tests
test = ks_tope(dataset)
```



The empirical test with a static baseline is not misspecified and detects emergence at the correct time, but

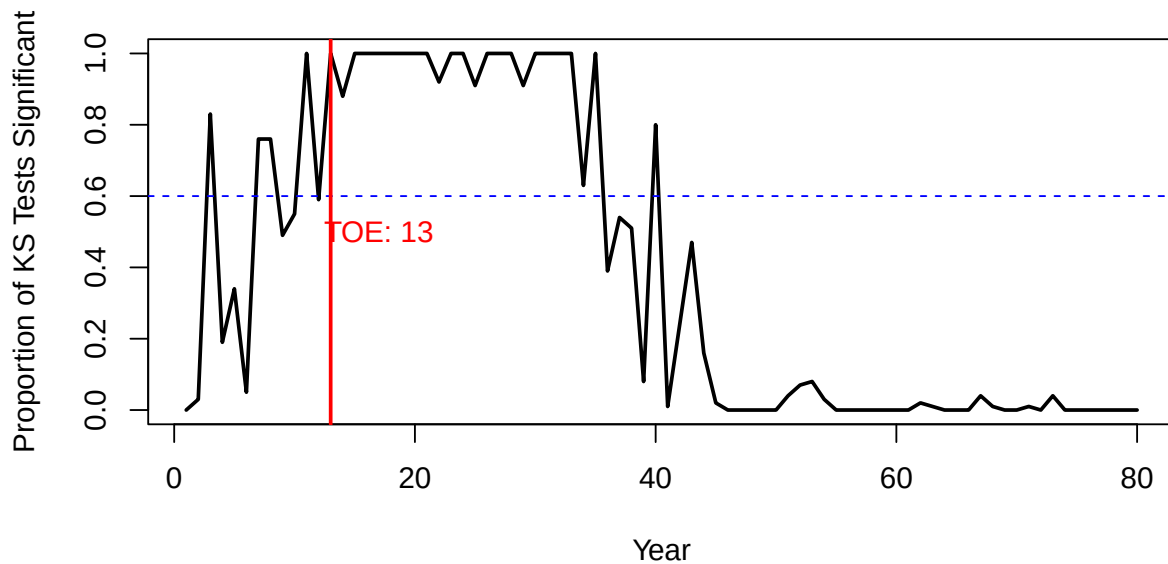
does not detect the return to a stationary state later in the time series:

```
test = emp_tope(dataset, max_y = 20, baseline = 'static')
```

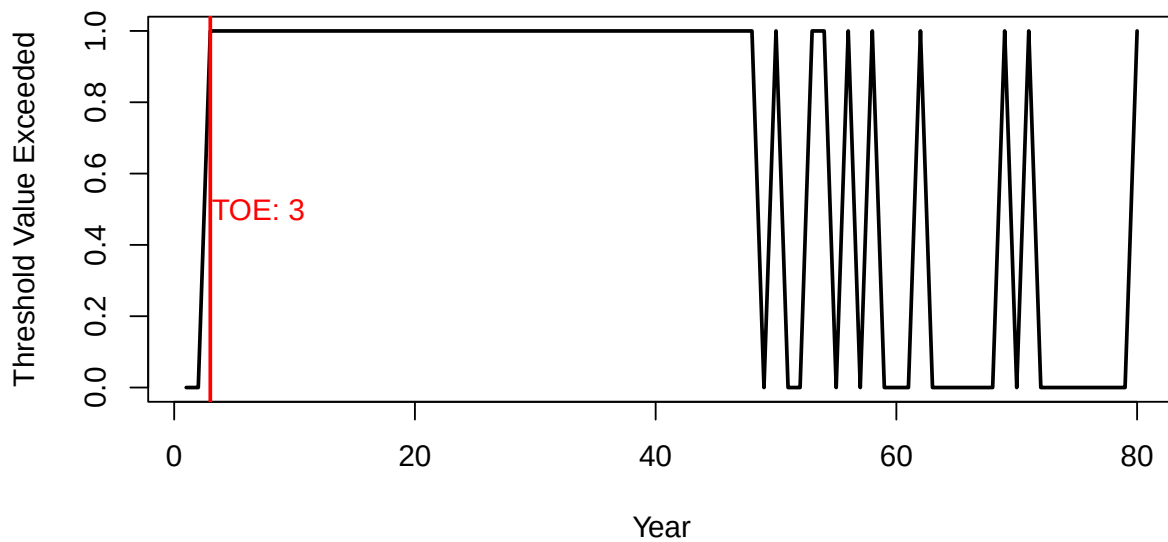


Using the moving window approach, both methodologies are able to detect both the original emergence in the first 30 years, and the subsequent return to a static state over the last 50. Note that both methods may be misspecified temporarily while the moving window contains both the changing and static state.

```
test = ks_tope(dataset, max_y = 20, unemergence = T)
```



```
test = emp_tope(dataset, max_y = 20, baseline = 'moving', unemergence = T)
```



The degree to which this occurs depends on how gradually or sharply the slope of the time series changes over time.

Full behaviour of empirical and statistical tests given different values of `max_y` and `baseline` are summarized below:

Method	max_y	baseline	behaviour
Empirical	0	'static'	Uses full time series to generate baseline
	>0	'static'	Uses first max_y years of the time series to generate baseline
	0	'moving'	Error
	>0	'moving'	Uses moving window of max_y years to generate baseline
Statistical	0	NA	Uses full time series to generate baseline
	>0	NA	Uses moving window of max_y years to generate baseline

2.4 Data Corrections and Considerations

When considering input data, corrections, and model assumptions, we recommend end-users conceptualize individual **phenometrics** ToE functions the same as they would linear models. Any changes made to raw data fed into ToE functions will be carried through ToE calculations and into the end result. End-users may leverage this to correct for issues in their data (e.g. bias, differences in uncertainty, etc.) as they would for a linear model. We provide some examples below that we believe may be generally applicable.

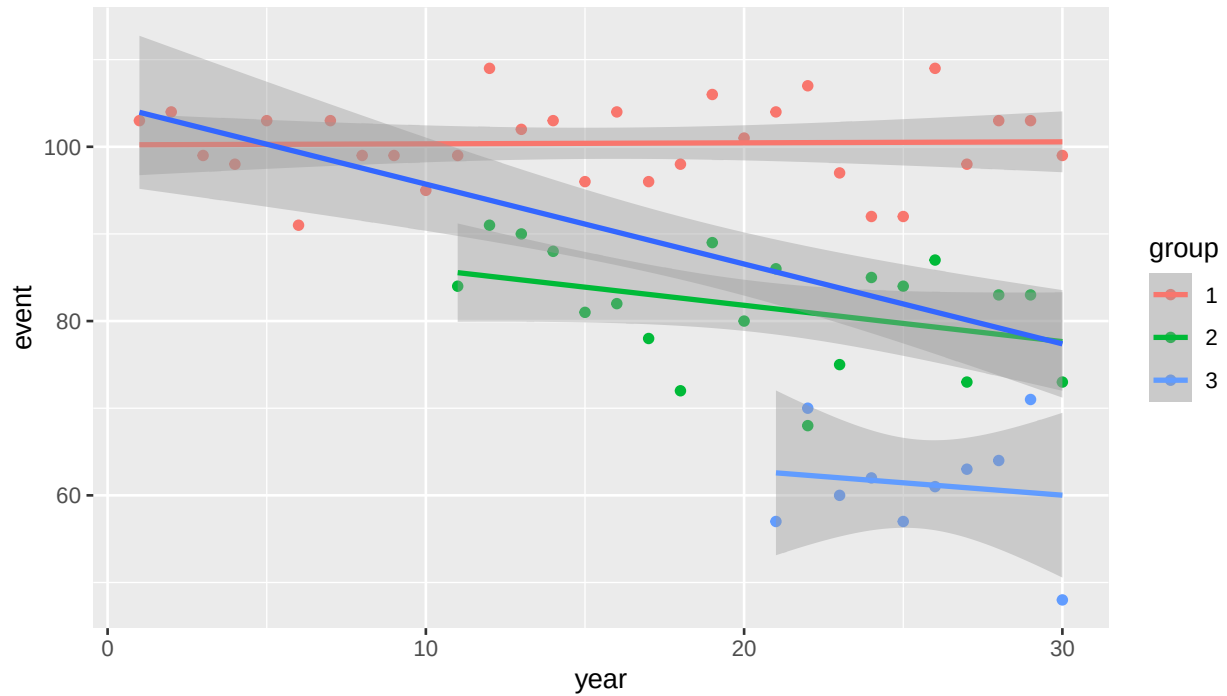
2.4.1 Group Bias

End-users who have grouped data may be subject to bias across groups, leading to emergence being incorrectly identified, or identified at the wrong time. For example, there might be systemic bias in sampling effort over time, or a species may shift its phenology at different rates throughout its range. In the example below, we have three groups with differing baselines, none of which have a phenological trend over time, which begin sampling at different times.

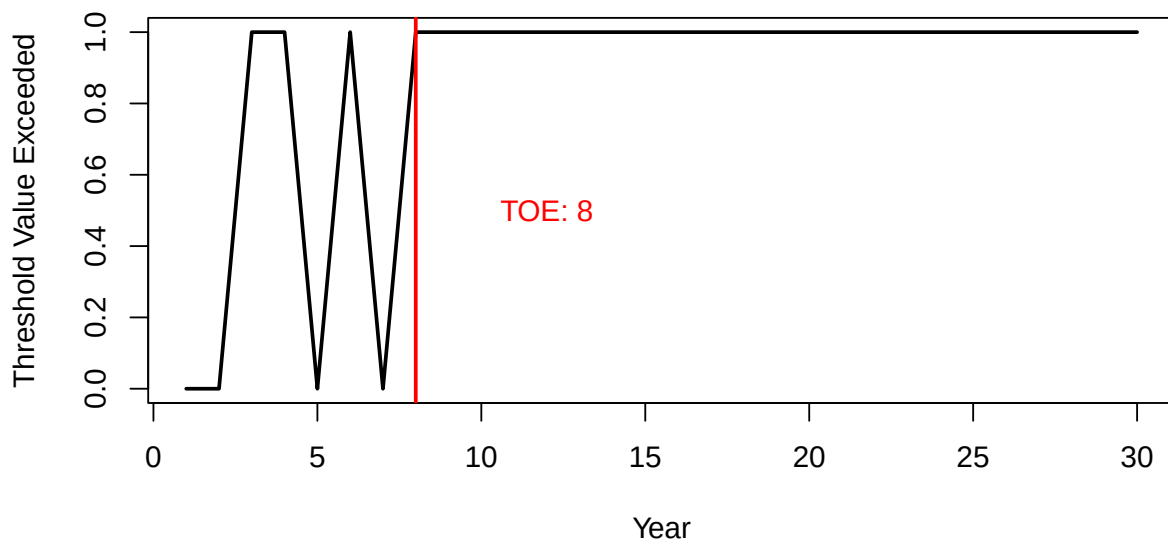
```
# Set seed
set.seed(12345)

# Create test dataset
g1 = data.frame(year = seq(1,30,1), event = round(rnorm(30, 100, 5)), group = '1')
g2 = data.frame(year = seq(11,30,1), event = round(rnorm(20, 80, 5)), group = '2')
g3 = data.frame(year = seq(21,30,1), event = round(rnorm(10, 60, 5)), group = '3')
dataset = rbind(g1,g2,g3)

# Plot
ggplot(dataset, aes(x = year, y = event, color = group)) +
  geom_point() + geom_smooth(method = 'lm') +
  geom_smooth(inherit.aes = F, data = dataset, aes(x = year, y = event), method = 'lm')
#> `geom_smooth()` using formula = 'y ~ x'
#> `geom_smooth()` using formula = 'y ~ x'
```



```
# Calculate empirical time of phenological emergence (ToPE)
default = emp_tope(dataset)
```

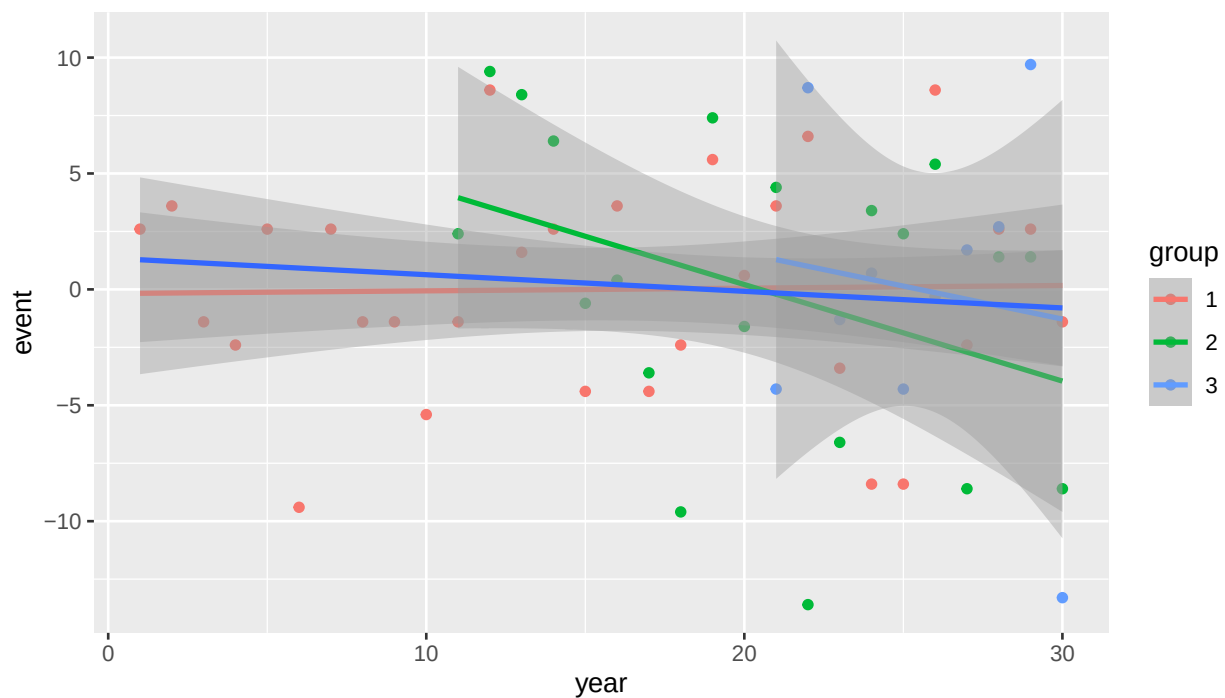


Despite no trend in any of these groups, emergence occurs because there is an overall trend when the data is pooled due to differences in each group's baseline and sampling period (Note the trend in the linear model of this dataset if group is not included). This can be corrected by adjusting the input data entered into the

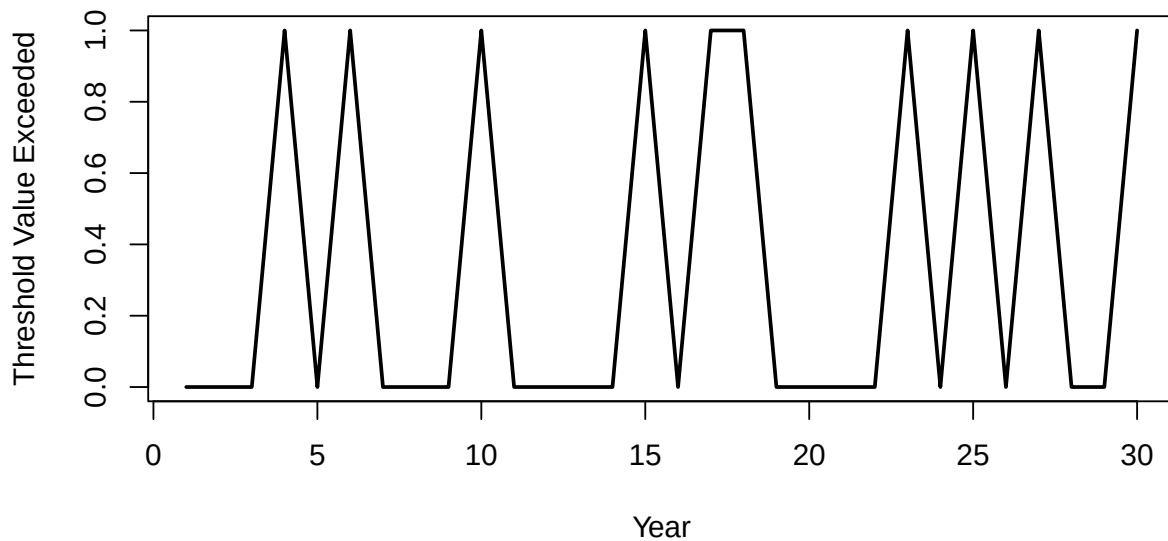
ToPE function. For example, we can run ToPE after performing group mean centering:

```
library(bruceR, quietly = T)
# Calculate group mean center
dataset$event_original = dataset$event
dataset$event = group_mean_center(dataset, vars = 'event', by = 'group')$event

# Plot
ggplot(dataset, aes(x = year, y = event, color = group)) +
  geom_point() + geom_smooth(method = 'lm') +
  geom_smooth(inherit.aes = F, data = dataset, aes(x = year, y = event), method = 'lm')
#> `geom_smooth()` using formula = 'y ~ x'
#> `geom_smooth()` using formula = 'y ~ x'
```



```
# Calculate empirical time of phenological emergence (ToPE)
default = emp_tope(dataset) # Emergence does not occur
#> Warning in min(which(emerged == 1)): no non-missing arguments to min; returning
#> Inf
#> Warning in min(which(emerged == 1)): no non-missing arguments to min; returning
#> Inf
#> Warning in min(which(emerged == 1)): no non-missing arguments to min; returning
#> Inf
```

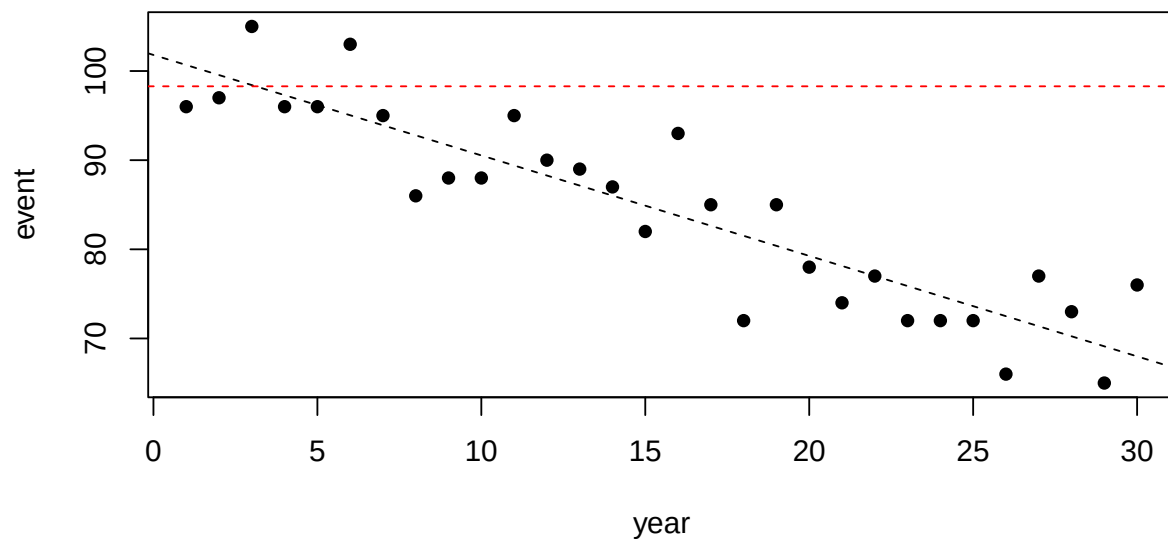


2.4.2 Weighting

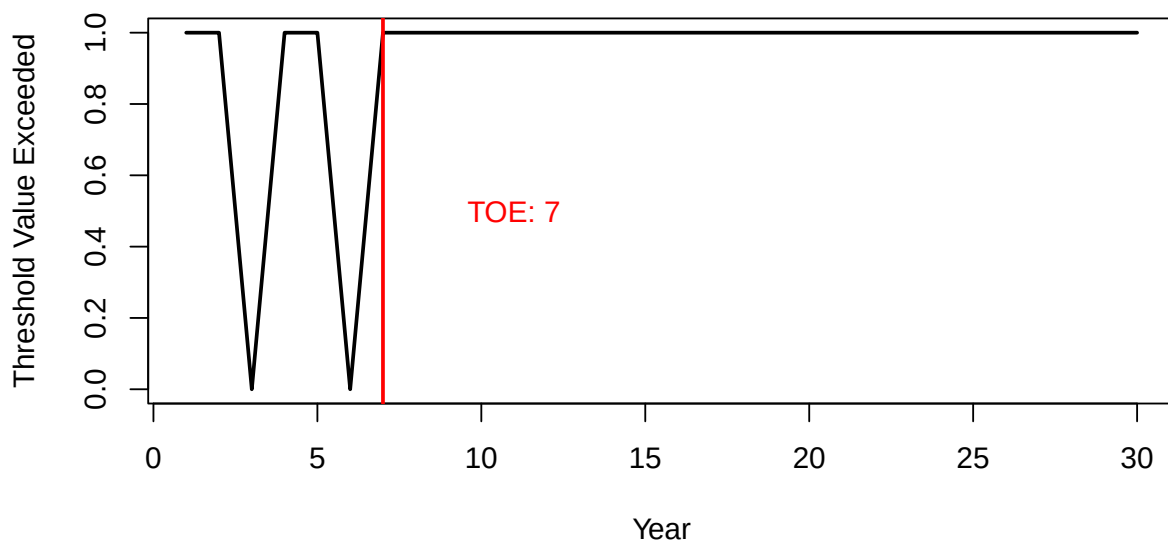
The ToE functions in `phenometrics` can carry any argument to `lm()`, with the exception of `formula` and `data`, which are fixed. One potential use of this functionality is to feed weights to the linear models inside the ToE functions, which are used to calculate the detrended counterfactual. This **does not** affect how each point is used in calculating test results, but it **does** affect how each point is used in calculating the baseline.

```
# Set seed
set.seed(123)

# Create test dataset
year = seq(1,30,1)
event = round(rnorm(30, 100, 5)) - seq(1,30,1)
dataset = data.frame(year, event)
plot(data = dataset, event ~ year, pch = 16)
abline(lm(data = dataset, event ~ year), lty = 'dashed')
abline(lm(data = dataset, event ~ year, weights = c(rep(1,7), rep(0,23))), col = 'red', lty = 'dashed')
```

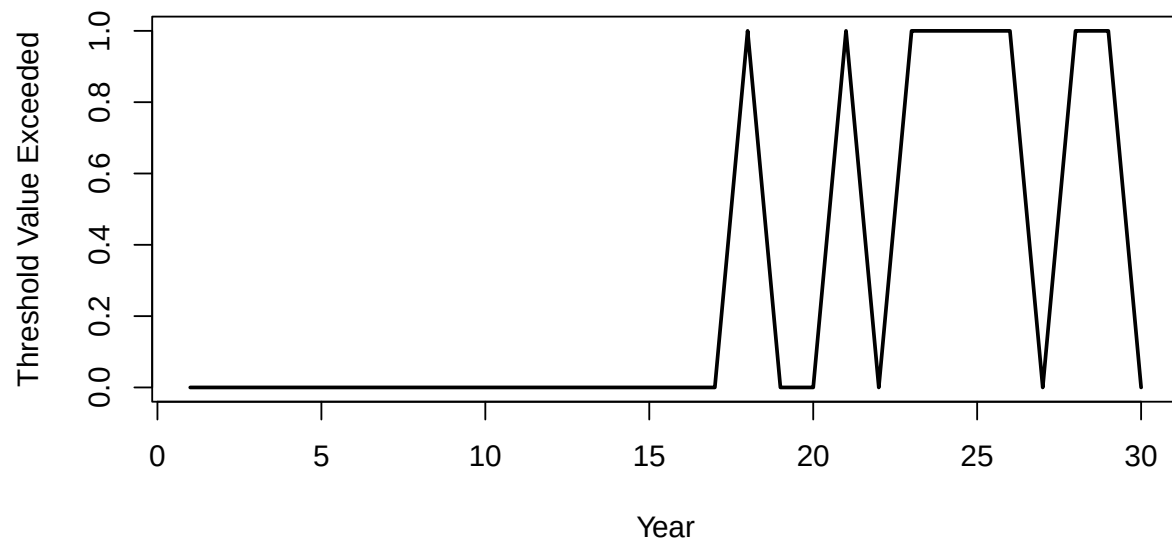



```
# Calculate empirical time of phenological emergence (ToPE)
default = emp_tope(dataset)
```



```
weighted = emp_tope(dataset, weights = c(rep(1,7), rep(0,23)))
#> Warning in min(which(emerged == 1)): no non-missing arguments to min; returning
#> Inf
```

```
#> Warning in min(which(emerged == 1)): no non-missing arguments to min; returning
#> Inf
#> Warning in min(which(emerged == 1)): no non-missing arguments to min; returning
#> Inf
```



In this (rather extreme) example, the red line is the weighted regression, and the black line is the unweighted regression. Users may use this functionality in the same way they would (or would not) for a standard linear model.

2.4.3 Resampling

By default, **phenometrics** does not calculate uncertainty in ToE values, as uncertainty in the dataset provided is baked into the ToE value itself - time series with greater uncertainty have later ToE values. One method by which end-users may calculate uncertainty in ToE values if they are interested in doing so is by resampling. For example, end-users may bootstrap each year in a dataset to generate a distribution of ToE values.

```
# Set seed
set.seed(123)

# Create test dataset
year = rep(seq(1,30,1),30)
event = round(rnorm(900, 100, 5)) - year
dataset = data.frame(year, event) %>% arrange(year)

em_boot = NULL
nboot = 1000

# bootstrap each year
```

```

for(i in 1:nboot){

  data_boot = NULL

  # bootstrap each year
  for(y in 1:length(dataset$year)){

    data_year = dataset[dataset$year == y,]
    data_year_boot = data_year[sample(1:nrow(data_year), nrow(data_year), replace = T),]
    data_boot = rbind(data_boot, data_year_boot)

  }

  # Run emergence test
  em = emp_tope(data_boot, plot = F)

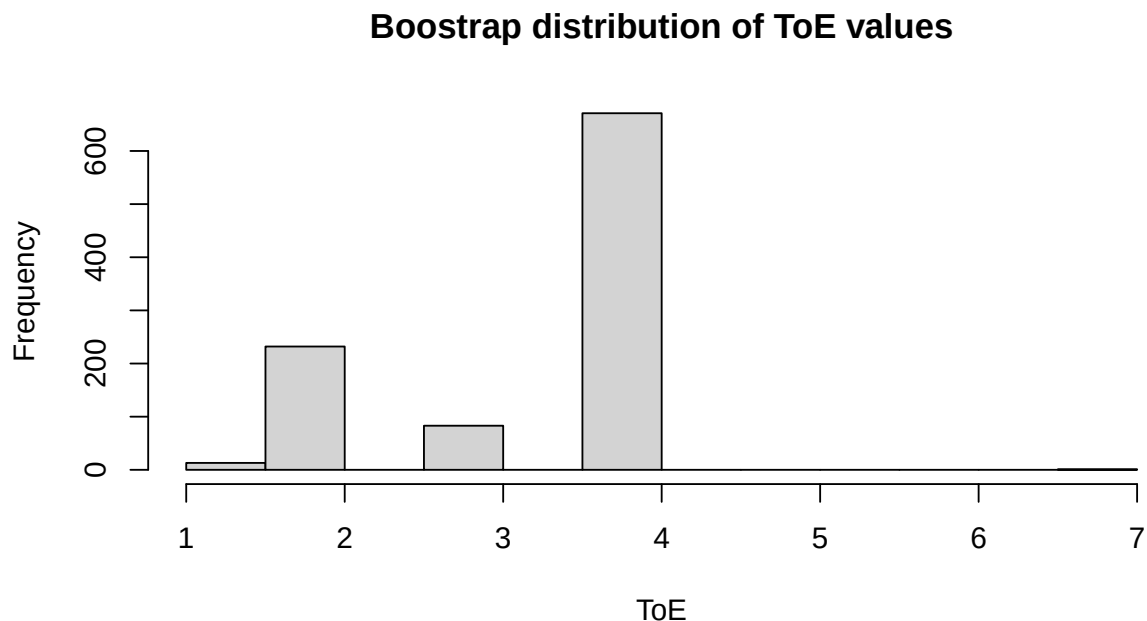
  # Calculate emergence year
  em_year = min(em[em$emerged == 1, 'year'], na.rm = T)

  # Concatenate result
  em_boot = c(em_boot, em_year)

}

# ToE Distribution
hist(em_boot, xlab = 'ToE', main = 'Bootstrap distribution of ToE values')

```



This provides a measure of how much ToE is affected by within-year sampling/measurement uncertainty.

2.4.4 Data gaps

Data gaps are not inherently problematic for **phenometrics** functions. They are accounted for automatically by linear regression in the generation of the counterfactual. However, users should note that **the `emt` parameter checks for consecutive years of data to quantify emergence, not a time range**. This means the presence of data gaps around the ToE may slightly shift the exact year of emergence.

As mentioned in the group bias section above, corrections may be applied to data before feeding into the **phenometrics** functions. If desired, this functionality may be used to fill gaps in the time series via the user's preferred method before calculating emergence

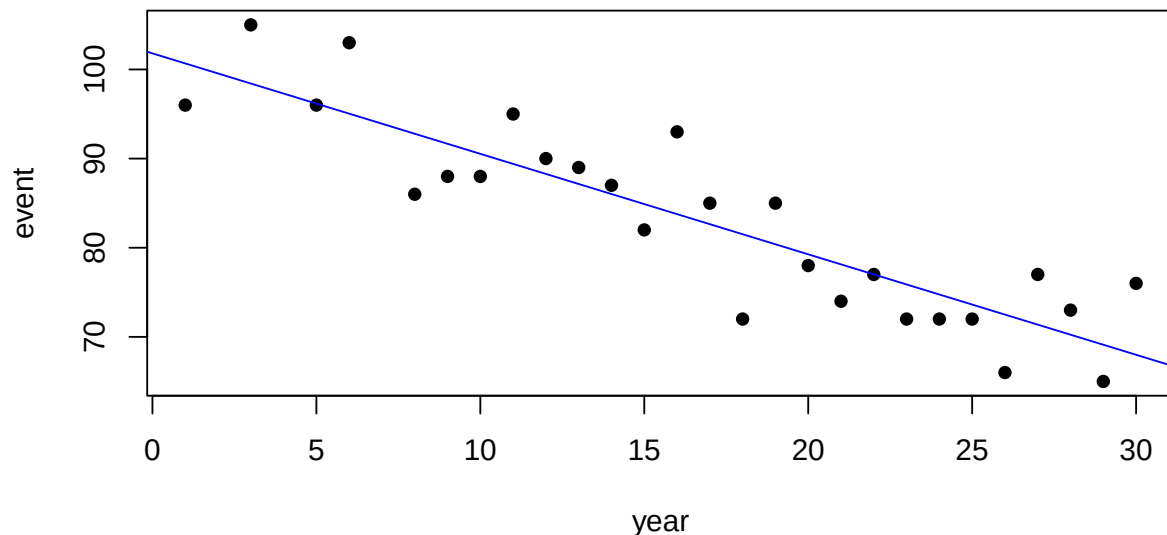
```
# Set seed
set.seed(123)

# Create test dataset
year = seq(1,30,1)
event = round(rnorm(30, 100, 5)) - seq(1,30,1)
dataset = data.frame(year, event)

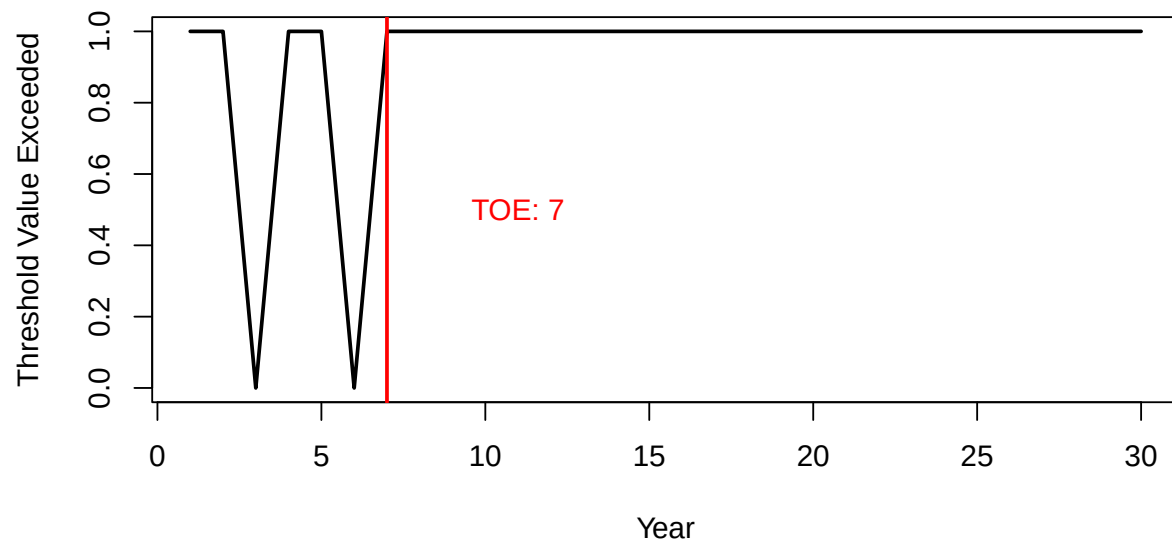
# Create gaps
dataset_gap = dataset[c(-2,-4,-7),]

# Fill gaps
gap_lm = lm(data = dataset, event ~ year)
dataset_filled = dataset
dataset_filled[c(2,4,7), 'event'] = predict(gap_lm, newdata = dataset[c(2,4,7),])

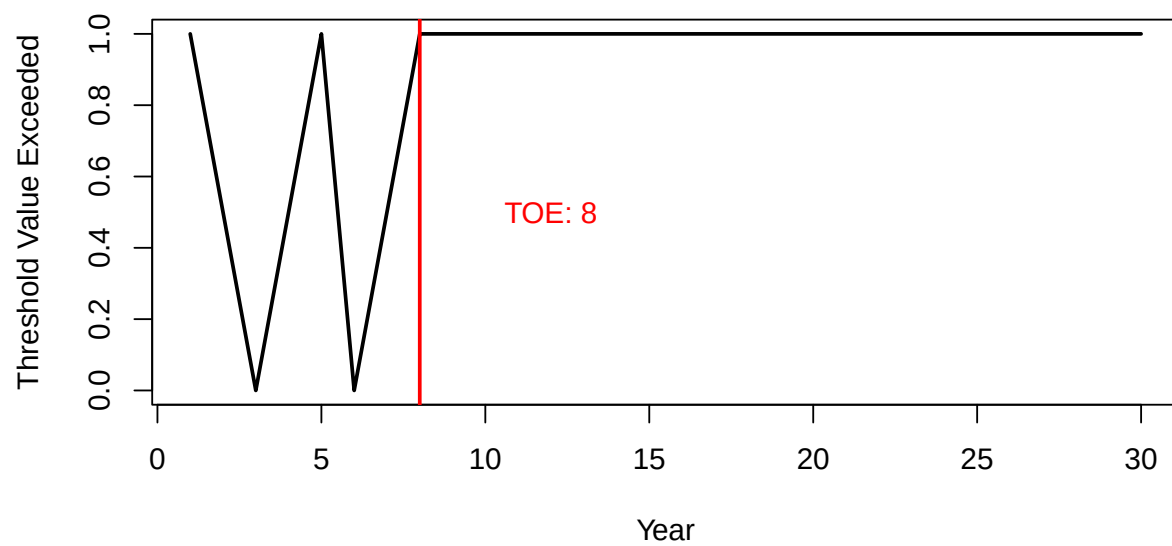
plot(data = dataset_gap, event ~ year, pch = 16)
abline(gap_lm, col = 'blue')
```



```
# Run tests
test = emp_tope(dataset)
```



```
test = emp_tope(dataset_gap)
```



```
test = emp_tope(dataset_filled)
```

